

InsightLens: A Vision-Language Based Accessibility Framework for Extracting and Understanding Classroom Content in Dual Languages

by

Syed Ar Rafi
24141215

Adiba Islam Khan
24141216

Ahsan Habib
22201027

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
Month Year.

© 2024. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Syed Ar Rafi

24141215

Adiba Islam Khan

24141216

Ahsan Habib

22201027

Approval

The thesis/project titled “InsightLens: A Vision-Language Based Accessibility Framework for Extracting and Understanding Classroom Content in Bangla and English Languages ” submitted by

1. Syed Ar Rafi (24141215)
2. Adiba Islam Khan (24141216)
3. Ahsan Habib (22201027)

of [Spring], [2026] has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in [Spring], [2026] .

Examining Committee:

Supervisor:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)

Md Tanzim Reza

Senior Lecturer
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor Chairperson
Department of Computer Science and Engineering
Brac University

Ethics Statement (Optional)

This is optional, if you don't have an ethics statement then omit this page

Abstract

Video-based learning has transformed education in Bangladesh, yet millions of technical lectures remain inaccessible as searchable, reviewable text due to the unique challenges of Banglish—the pervasive code-mixing of Bengali and English in educational discourse. This thesis presents a Multimodal Banglish Classroom Summarizer, an end-to-end system that transforms lecture videos into structured notes by fusing automatic speech recognition with visual content extraction. The system employs OpenAI’s Whisper for speech transcription, Qwen2.5-VL-7B for whiteboard keyword extraction, and cross-modal verification to correct ASR errors using visual ground truth. We introduce the BanglaASR benchmark dataset comprising nine technical lectures (73,141 characters) across Python programming, Digital Logic Design, and Database Management Systems, along with term-based evaluation metrics that accommodate the orthographic variability inherent to Romanized Banglish. Experimental evaluation demonstrates 73.9% Term F1, 83.6% Term Precision, and 66.5% Term Recall, with cross-modal fusion improving baseline Whisper performance by 5.7 percentage points. Our analysis reveals that verification-oriented fusion outperforms naive visual biasing, which introduces hallucinations that degrade accuracy. The system operates on consumer-grade GPU hardware (NVIDIA RTX 3090, 24GB VRAM) through careful memory management. This work establishes reproducible baselines for Banglish technical lecture processing and contributes both methodological insights and practical tools for an underserved linguistic community of over 230 million Bengali speakers.

Keywords: Automatic Speech Recognition, Code-Mixed Language Processing, Banglish, Multimodal Learning, Vision-Language Models, Lecture Transcription, Educational Technology

Dedication (Optional)

A dedication is the expression of friendly connection or thanks by the author towards another person. It can occupy one or multiple lines depending on its importance. You can remove this page if you want.

Acknowledgement

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Introduction

The digital revolution in education has fundamentally reshaped how knowledge flows from instructors to learners across the globe. What was once confined to physical classrooms and printed textbooks now travels instantly through fiber-optic cables and wireless networks, reaching students in remote villages and bustling cities alike. Video-based learning platforms, massive open online courses, and recorded classroom lectures have emerged as powerful equalizers in the educational landscape, offering unprecedented access to quality instruction regardless of geographical or socioeconomic constraints. Yet beneath this promise of democratized education lies a persistent challenge: the vast majority of video content remains locked in its audio-visual form, inaccessible to those who cannot hear, difficult to search for those seeking specific concepts, and time-consuming for those who learn better through reading than watching.

This thesis confronts this challenge head-on by developing a multimodal artificial intelligence system capable of automatically transforming classroom lecture videos into comprehensive, well-structured lecture notes. The system we propose does not merely transcribe speech—it understands context, extracts meaning from visual cues, and synthesizes information from multiple sources to produce output that genuinely serves student learning needs.

At the heart of our research lies a linguistic phenomenon that makes this problem particularly fascinating and technically demanding: **Banglish**. This term describes the natural code-mixing that occurs when Bengali speakers, particularly in educational contexts, seamlessly weave English words and phrases into Bengali sentence structures. A computer science instructor in Dhaka does not pause to switch languages when explaining object-oriented programming; instead, they might say “*Amra class theke object create korbo*”—a single fluid utterance meaning “We will create an object from a class,” where English technical terms like “class,” “object,” and “create” nestle comfortably within Bengali grammatical frameworks. This linguistic fluidity, while natural and effective for human communication, poses formidable challenges for automatic speech recognition systems designed with monolingual assumptions.

Our approach recognizes that classroom lectures are inherently multimodal experiences. Students do not learn solely by listening; they watch the instructor write on the whiteboard, observe diagrams being constructed, and connect verbal ex-

planations to visual representations. We harness this multimodal richness by processing both the audio stream—capturing the instructor’s spoken words—and the visual stream—extracting text, keywords, and structural information from whiteboard content. When the instructor writes “Object-Oriented Programming” on the whiteboard, that visual text serves as unambiguous ground truth, even if the audio transcription garbles the spoken phrase. By intelligently fusing information from both modalities, our system achieves transcription accuracy that neither modality could accomplish in isolation.

1.2 Background

1.2.1 The Rise of Video-Based Learning

The transformation of education through video content represents one of the most significant shifts in pedagogical history. What began as simple lecture recordings has evolved into a sophisticated ecosystem of educational media that spans the globe. Platforms such as YouTube, Coursera, edX, and Khan Academy have collectively amassed billions of hours of instructional content, covering subjects from elementary arithmetic to advanced quantum mechanics. This proliferation has been particularly transformative in developing nations, where traditional educational infrastructure often struggles to meet the demands of growing populations hungry for knowledge. In Bangladesh, the impact of video-based learning has been nothing short of revolutionary. Platforms like 10 Minute School and Shikho have become household names, serving millions of students preparing for board examinations, university admissions, and competitive assessments. Individual educators have built substantial followings on YouTube, delivering lectures on everything from HSC physics to university-level computer science. For students in rural areas, where qualified teachers for advanced subjects may be scarce, these video resources represent a lifeline—a window into educational opportunities previously accessible only to their urban counterparts. Yet despite this remarkable proliferation, a fundamental accessibility gap persists. Consider a student with hearing impairment attempting to learn from a video lecture: without accurate captions, the content remains entirely inaccessible. Consider a student preparing for an examination who needs to quickly review a specific concept: scrubbing through a forty-minute video searching for a two-minute explanation wastes precious study time. Consider a student who processes information more effectively through reading than listening: the video format forces them into a sub-optimal learning modality. These scenarios illustrate a crucial insight: while video content has democratized access to instruction, the format itself creates barriers that text-based materials do not.

Automatic lecture summarization emerges as a powerful solution to these challenges. By converting the ephemeral spoken word into persistent, searchable, and skimmable text, we can unlock the full potential of video-based education. Students can search for specific terms, jump directly to relevant sections, review key concepts quickly, and access content regardless of hearing ability. The challenge, of course, lies in performing this conversion accurately—particularly when the spoken language defies the assumptions built into most speech recognition systems.

1.2.2 Automatic Speech Recognition: Current State

The field of automatic speech recognition has witnessed remarkable progress over the past decade, driven primarily by deep learning advances and the availability of massive training datasets. OpenAI’s Whisper model **Radford2023**, released in 2022 and subsequently refined through multiple versions, represents the current state of the art for multilingual speech recognition. Trained on approximately 680,000 hours of labeled audio data spanning nearly 100 languages, Whisper achieves transcription accuracy that approaches human performance on clean English speech. Commercial offerings from Google, Amazon, and Microsoft have similarly matured, powering voice assistants, captioning services, and transcription tools used by millions daily. However, this impressive progress has not benefited all languages equally. The machine learning community has long recognized a hierarchy of language resources **Conneau2021**: at the top sit “high-resource” languages like English, Mandarin, and Spanish, blessed with abundant training data, extensive research attention, and robust commercial support. At the bottom languish “low-resource” languages, often spoken by millions yet starved of the digital resources necessary for effective machine learning. Bengali occupies a curious middle ground in this hierarchy. As the seventh most spoken language globally, with over 230 million native speakers, Bengali commands significant demographic weight. Yet when it comes to technical content transcription—particularly in the code-mixed Banglish variety common in educational settings—the available resources and research attention fall far short of what the speaker population might suggest.

The challenges facing ASR systems when processing Banglish technical lectures are manifold. First and most fundamentally, code-mixing confounds models trained on monolingual data. A Whisper model optimized for English may handle the English portions adequately but will stumble on Bengali words, often producing phonetically similar but semantically nonsensical English substitutions. Conversely, a Bengali-focused model like BanglaASR outputs text in Bengali Unicode script, ignoring English terms entirely and rendering the output difficult to compare with Romanized ground truth.

Second, technical vocabulary presents persistent challenges. Domain-specific terms like “polymorphism,” “database normalization,” or “flip-flop circuits” appear rarely in general-purpose training corpora. When an instructor pronounces these terms with a Bengali accent, or embeds them within Bengali grammatical structures (“polymorphism ta important”), even sophisticated models may fail to recognize them correctly.

Third, acoustic conditions in real classroom recordings differ substantially from the clean studio audio that dominates ASR training sets. Background noise from fans and air conditioners, reverberation from hard classroom walls, varying distances between speaker and microphone, and occasional overlapping speech from student questions all degrade recognition accuracy. These acoustic challenges compound the linguistic difficulties, creating a recognition environment far more challenging than ASR benchmarks typically assume.

Finally, the absence of standardized Romanization for Bengali creates evaluation difficulties that ripple throughout the research process. Unlike languages with established transliteration schemes, Bengali written in Roman script admits multiple valid spellings for the same word. The first-person plural pronoun, for instance, might be Romanized as “amra,” “aamra,” “amraa,” or even “amara”—all repre-

senting identical pronunciation and meaning. This variability means that traditional evaluation metrics like Word Error Rate, which count exact string matches, systematically underestimate transcription quality for Romanized Banglish.

1.2.3 Multimodal Learning and Cross-Modal Fusion

The past three years have witnessed an explosion of interest in vision-language models—systems capable of understanding and reasoning about images using natural language **Radford2021**, **Liu2023LLaVA**, **Wang2024Qwen2VL**. Models such as GPT-4V, LLaVA, Qwen2.5-VL, and numerous others can describe image contents, answer complex questions about visual scenes, extract and interpret text from photographs and documents, and even generate images from textual descriptions. These capabilities, unimaginable just five years ago, have opened new frontiers in artificial intelligence research and application.

For lecture processing, vision-language models offer a tantalizing opportunity. Classroom instruction is inherently multimodal: students do not simply listen to lectures; they watch instructors write on whiteboards, project slides, draw diagrams, and gesture toward visual aids. This visual channel carries information that complements and sometimes clarifies the audio channel. When an instructor writes “Object-Oriented Programming” on the whiteboard while explaining the concept verbally, the visual text serves as unambiguous ground truth for the terminology being discussed. Even if acoustic conditions or pronunciation variations cause the ASR system to mishear “Object-Oriented Programming” as “object ornamental programming,” the visual extraction can provide the correct term.

This observation—that whiteboard content serves as reliable ground truth for technical terminology—forms a cornerstone of our approach. By extracting text from video frames using vision-language models, we obtain a vocabulary of terms that the instructor considers important enough to write down. These terms can then be used to guide, verify, or correct the ASR output. If the visual extraction identifies “polymorphism” as a whiteboard keyword, and the ASR output contains “poly more fizz him” at a corresponding timestamp, we have strong evidence for a correction opportunity.

Cross-modal fusion, however, is not straightforward. The audio and visual channels operate on different timescales and carry different types of information. The instructor may write a term on the whiteboard minutes before or after discussing it verbally. The visual channel may contain historical content—previous concepts still visible on the board—alongside currently relevant material. Naively biasing ASR toward all visually-extracted keywords can introduce errors, causing the system to hallucinate keyword mentions where none actually occur. Our research explores these fusion challenges, investigating when and how cross-modal information should be combined for maximum benefit.

1.2.4 The Banglish Challenge

Banglish represents far more than simple language alternation; it embodies a sophisticated linguistic strategy that reflects the educational and cultural realities of contemporary Bangladesh **MyersScotton1993**, **Muysken2000**. Unlike code-switching, where speakers alternate between languages at sentence or clause bound-

aries, code-mixing involves the intimate integration of two linguistic systems at the word, phrase, and even morphological levels. This distinction matters profoundly for natural language processing: while code-switching might be handled by language identification followed by monolingual processing, code-mixing demands truly bilingual understanding at the finest granularities.

The examples from our dataset illuminate this complexity. When an instructor says “*Python e variable declare korte hole eivabe korbo*” (“To declare a variable in Python, we do it this way”), the English terms “Python,” “variable,” and “declare” are not merely inserted into a Bengali sentence—they are morphologically integrated into Bengali grammatical structures. The locative suffix “-e” attaches to “Python” to form “Python e” (in Python), treating the English proper noun as a Bengali nominal. Similarly, “variable declare korte” combines the English noun-verb pair with the Bengali infinitive marker “korte,” creating a hybrid verbal construction that neither language would produce in isolation.

This morphological integration poses severe challenges for tokenization and recognition. Standard English ASR models may recognize “Python” but will likely misinterpret “e” as a separate word or discard it as noise. Bengali models may handle the suffix correctly but fail to recognize “Python” as a meaningful unit. The resulting transcriptions fragment words unpredictably, creating outputs that serve neither English nor Bengali readers well.

Consider the challenge from another angle: evaluation. How should we assess whether a transcription of “*SQL e SELECT query diye data retrieve kora hoy*” is correct? If the system produces “SQL e SELECT query diye data retrieve kora hoi,” is this an error? The final word differs (“hoy” versus “hoi”), but both are valid Romanizations of the same Bengali pronunciation. Traditional Word Error Rate would count this as an error; a more linguistically informed metric would recognize it as correct. Our research develops and validates such metrics, contributing evaluation methodology alongside technical systems.

1.3 Rationale of the Study

The motivation for this research emerges from a convergence of technological opportunity, educational necessity, and scholarly responsibility to address underserved linguistic communities.

1.3.1 Educational Equity and Access

Education in Bangladesh, like many developing nations, remains marked by significant inequalities. Students in Dhaka enjoy access to renowned coaching centers, experienced tutors, and well-equipped schools; their counterparts in rural Sylhet or Rangpur often make do with understaffed schools and limited resources. The digital revolution, and particularly the rise of video-based learning, has begun to address this imbalance. A student in the most remote village can now access the same lecture that a student in the capital watches—provided they have an internet connection and a smartphone.

Yet this democratization remains incomplete. Video content, by its nature, demands sustained attention and linear consumption. A student cannot skim a video the way they might skim a textbook, quickly identifying relevant sections and skipping

material they already understand. Searching for a specific concept within a video requires scrubbing through the timeline, hoping to land near the relevant segment. For students juggling academic responsibilities with work or family obligations, this inefficiency translates to hours of wasted time.

Accurate, searchable lecture transcriptions address these limitations directly. With text-based notes, students can use Ctrl+F to locate specific terms instantly. They can skim headings and topic sentences to gauge relevance before committing attention. They can copy key definitions or code snippets into their own notes without pausing to transcribe manually. For students with hearing impairments, text-based content transforms inaccessible videos into fully usable learning resources.

The system we develop in this thesis aims to provide these benefits automatically, at scale, for the Banglish technical lectures that serve millions of Bangladeshi students. By converting video lectures into structured notes, we contribute to a more equitable educational landscape where all students—regardless of disability, learning style preference, or time constraints—can benefit from the wealth of instructional content available online.

1.3.2 Addressing a Research Gap

Beyond practical impact, this research addresses a significant gap in the scholarly literature. Despite the global importance of code-mixed languages—spoken by hundreds of millions in South Asia, Southeast Asia, Africa, and diaspora communities worldwide—research attention has concentrated overwhelmingly on monolingual scenarios. English dominates ASR research; when multilingual systems are considered, they typically address language identification and switching rather than the intimate code-mixing characteristic of Banglish.

Our literature review, detailed in Chapter 2, revealed a striking absence of foundational resources. No benchmark dataset exists for Banglish technical lecture transcription; researchers wishing to work on this problem have no standard corpus against which to evaluate their systems. No published work addresses multimodal fusion specifically for Banglish ASR enhancement; the techniques developed for English or Mandarin may not transfer effectively to this linguistically distinct context. Limited research explores term-based evaluation metrics appropriate for code-mixed languages; the standard metrics inherited from monolingual ASR research penalize valid transliteration variants as errors.

This thesis contributes to filling these gaps. We create a benchmark dataset of nine classroom recordings with manually transcribed ground truth, providing future researchers with evaluation resources. We develop and test multimodal fusion techniques specifically designed for the Banglish context, documenting both successful approaches and instructive failures. We propose and validate evaluation metrics that account for transliteration variance and code-mixing phenomena, advancing the methodology available to the research community.

1.3.3 Practical Demand and User Needs

The motivation for this research is not purely academic; it responds to genuine user needs that we have observed firsthand. Students in Bangladeshi universities routinely express frustration with the manual effort required to extract value from

video lectures. They describe spending hours transcribing key portions of recordings, pausing and rewinding repeatedly to catch technical terms. They share hand-written notes that inevitably contain gaps and errors. They ask whether any tool exists to automate this tedious process.

This demand reflects a broader pattern: the gap between available technology and user needs in underserved linguistic contexts. While English speakers can choose from dozens of transcription services—from YouTube’s automatic captions to dedicated tools like Otter.ai—Banglish speakers have few options. The tools that exist handle monolingual Bengali or English; the code-mixed reality of Bangladeshi technical education falls through the cracks.

By developing a system that actually works for Banglish—acknowledging and accommodating its linguistic complexity rather than treating it as noise to be filtered—we respond to this unmet demand. The practical impact extends beyond convenience to genuine educational benefit: students who spend less time on mechanical transcription have more time for conceptual understanding, problem-solving, and creative learning.

1.4 Problem Statement

This research addresses a central question that sits at the intersection of speech recognition, computer vision, natural language processing, and educational technology:

How can multimodal artificial intelligence effectively process Banglish technical lecture videos to generate accurate, structured lecture notes, given the challenges of code-mixing, ASR errors, and limited training resources?

This seemingly simple question conceals layers of technical complexity. Each component of the problem—“multimodal,” “Banglish,” “technical lectures,” “accurate,” “structured”—introduces constraints and challenges that must be carefully navigated. We decompose this overarching problem into five specific technical challenges, each of which demands focused attention within our research.

1.4.1 Challenge 1: Accurate Speech Recognition for Banglish

The foundational challenge of this research lies in obtaining accurate transcriptions from Banglish speech. Current ASR systems, regardless of their sophistication on well-resourced languages, struggle substantially with Banglish technical content. This struggle manifests in several interconnected ways.

When Whisper, the current state-of-the-art multilingual model, processes Banglish audio, it tends to interpret everything through an English lens. Bengali words are approximated by phonetically similar English words, producing outputs that may sound vaguely correct when read aloud but carry entirely different meanings. The Bengali word “kori” (meaning “I do”) might become “curry”; “dekhbo” (meaning “I will see”) might become “decade.” These substitutions accumulate across an entire lecture, rendering the transcript unusable for study purposes.

Bengali-focused models like BanglaASR face the opposite problem: they handle Bengali portions reasonably well but stumble on English technical terms. When an instructor says “Let’s discuss the SELECT statement,” a Bengali model might produce nonsensical Bengali words that approximate the English sounds, or simply omit the unfamiliar terms entirely. Furthermore, BanglaASR outputs text in Bengali Unicode script (৳), creating a script mismatch with the Romanized Banglish that students and our evaluation framework expect.

Technical vocabulary compounds these difficulties. Terms like “polymorphism,” “recursion,” or “normalization” appear rarely in general ASR training data. When pronounced with a Bengali accent or embedded within Bengali grammatical structures, recognition accuracy plummets. The instructor might say “polymorphism ta understand korte hole”—a perfectly natural Banglish construction—but ASR systems trained on clean English or Bengali monolingual data have no framework for interpreting this hybrid utterance.

Acoustic conditions in real classroom recordings add another layer of difficulty. Unlike the studio-quality audio that dominates ASR benchmarks, classroom recordings feature reverberant acoustics, background noise from air conditioning or fans, variable speaker-to-microphone distances, and occasional interruptions from student questions. These acoustic challenges would degrade performance even for well-supported languages; for Banglish, they transform an already difficult problem into a formidable one.

1.4.2 Challenge 2: Cross-Modal Information Extraction

The visual channel of lecture videos—particularly whiteboard content—offers a potential solution to ASR limitations. When the instructor writes “Object-Oriented Programming” on the whiteboard, that visual text is unambiguous and correctly spelled, regardless of how the spoken version might be transcribed. Extracting and leveraging this visual information, however, presents its own set of challenges.

Classroom video quality varies enormously. Some recordings feature high-definition cameras focused on well-lit whiteboards; others capture handheld smartphone footage of dimly lit classrooms from unfavorable angles. Lighting conditions change as instructors move, cast shadows, or partially occlude the board. These variations challenge even sophisticated vision-language models, which may confidently extract incorrect text or miss content entirely under adverse conditions.

Handwritten text recognition, while it has improved dramatically with modern deep learning approaches, remains more challenging than printed text recognition. Instructors have idiosyncratic handwriting styles; they may write quickly, use inconsistent spacing, or employ abbreviations that make sense in context but confuse automated systems. A hastily written “OOP” on the whiteboard might be misread as “COP” or “00P” by a vision model unfamiliar with the educational context.

Temporal alignment between visual and audio content presents additional complexity. The instructor may write a term on the whiteboard and only explain it verbally several minutes later—or may explain a concept first and then write the key term. Historical content remains visible on the board even after the discussion has moved to new topics. Distinguishing between currently relevant visual content and historical artifacts requires understanding the lecture’s progression over time, a task that demands more than simple frame-by-frame analysis.

1.4.3 Challenge 3: Multimodal Fusion Strategy

Given information from both audio and visual channels, how should a system combine them to produce optimal output? This question of multimodal fusion lies at the heart of our research, and it admits no simple answer.

The most straightforward approach—biasing ASR toward visually-extracted keywords—carries significant risks. If the visual extraction identifies fifty keywords from whiteboard content, and we boost the probability of these keywords throughout the transcription process, we may cause the system to hallucinate keyword mentions where none actually occur. The word “class” might be inserted dozens of times simply because it appears prominently on the whiteboard, even when the instructor is discussing unrelated topics.

Verification-based approaches, where visual content is used to check rather than guide ASR outputs, face different challenges. How confident must we be in a visual extraction before using it to override an ASR decision? What if the visual and audio channels genuinely disagree—the instructor says one thing but has written something different on the board? How do we handle partial matches, where the ASR produces “object orient programming” and the visual extraction shows “Object-Oriented Programming”?

Temporal alignment adds another dimension to fusion decisions. A keyword extracted from a frame at the five-minute mark should probably influence transcription around that timestamp, but how wide should the influence window be? Keywords written early in the lecture may remain relevant throughout; keywords written during specific explanations may only be relevant locally. Determining appropriate temporal scopes for cross-modal influence requires understanding both the visual persistence of whiteboard content and the thematic structure of spoken explanations.

1.4.4 Challenge 4: ASR Hallucination Detection and Correction

A particularly insidious failure mode of ASR systems, especially when processing challenging audio, is hallucination: the generation of text that has no correspondence to actual speech. Unlike simple transcription errors, where the system mishears one word as another, hallucinations involve the fabrication of entire phrases or the excessive repetition of words or patterns.

In our early experiments, we observed hallucinations taking several characteristic forms. Repetitive hallucinations involve the same word or short phrase repeated dozens or hundreds of times: “the the the the the the” or “class class class class.” These repetitions can inflate transcripts enormously, burying genuine content under masses of hallucinated text. Phrase hallucinations involve the insertion of stock phrases that appear in ASR training data but bear no relation to the actual lecture: “thank you for watching,” “subscribe to our channel,” or similar artifacts from YouTube-sourced training data.

These hallucinations severely degrade transcript utility. A student reviewing a transcript expects it to represent what the instructor actually said; encountering pages of repetitive nonsense or irrelevant phrases destroys trust in the output and renders the transcript useless for study purposes. Detecting and removing hallucinations with-

out inadvertently removing legitimate content poses a significant technical challenge that we address through dedicated post-processing mechanisms.

1.4.5 Challenge 5: Evaluation Without Established Benchmarks

The final challenge concerns not system development but system evaluation. How do we know whether our approach is working? This question, seemingly straightforward for well-established tasks, becomes surprisingly difficult for Banglish technical lecture transcription.

Traditional ASR evaluation relies heavily on Word Error Rate (WER) **Jurafsky2000**: the edit distance between predicted and reference transcriptions, normalized by reference length. This metric assumes that there exists a single correct transcription against which predictions should be judged. For Romanized Banglish, this assumption fails. The same spoken utterance can be validly transcribed in multiple ways, depending on Romanization conventions. If the instructor says the Bengali word for “we,” the reference might be “amra,” but “aamra,” “amraa,” and “amara” are all equally valid Romanizations. A system that produces “aamra” when the reference says “amra” would be penalized under WER, despite having transcribed correctly. Beyond Romanization variance, no standard benchmark dataset exists for Banglish technical lectures. Researchers working on English ASR can evaluate against LibriSpeech, CommonVoice, or numerous other established corpora; researchers working on Banglish technical lectures must create their own evaluation resources from scratch. This requirement introduces substantial overhead and makes comparison across studies difficult.

Our research addresses this challenge by creating ground truth transcriptions for our dataset and developing evaluation metrics that account for Romanization variance. We propose term-based metrics—Term Recall, Term Precision, and Term F1—that focus on whether technical keywords are correctly captured rather than whether every word matches exactly. These metrics better reflect the practical utility of transcriptions for student learning while accommodating the linguistic variability inherent to Romanized Banglish.

1.5 Research Objectives

Having articulated the problem and its constituent challenges, we now state the specific objectives that guide this research. These objectives are organized into primary goals—those essential to the thesis’s core contribution—and secondary goals that extend and enrich the primary work.

1.5.1 Primary Objectives

Objective 1: Develop a Multimodal Lecture Processing Pipeline. The first and most fundamental objective is to create a complete, functional system that processes Banglish lecture videos from raw input to structured output. This pipeline must ingest video files, extract audio for speech recognition, sample frames for visual analysis, perform both ASR and visual text extraction, fuse information from multiple modalities, and generate coherent lecture notes suitable for student

use. The system must be modular, allowing individual components to be improved or replaced as technology advances, and must operate within reasonable computational constraints on available hardware.

Objective 2: Design Cross-Modal Verification Mechanisms. The second objective focuses on leveraging the complementary nature of audio and visual information. We aim to develop techniques that use whiteboard content to verify ASR outputs, identifying likely errors where the audio transcription diverges from visual evidence. Beyond verification, we explore whether visual information can guide correction, substituting visually-confirmed terms for acoustically-garbled alternatives. This objective requires careful attention to the risks of over-correction and the establishment of appropriate confidence thresholds.

Objective 3: Create a Benchmark Dataset. The third objective addresses the absence of evaluation resources for Banglish technical lectures. We aim to create a dataset of real classroom recordings with manually transcribed ground truth, covering multiple technical domains and instructional styles. This dataset serves dual purposes: enabling rigorous evaluation of our own system and providing a resource for future researchers working on similar problems. The ground truth transcriptions must follow consistent Romanization conventions while accommodating the inherent variability of Banglish.

Objective 4: Propose and Validate Appropriate Evaluation Metrics. The fourth objective concerns evaluation methodology. Recognizing that traditional WER penalizes valid Romanization variants as errors, we aim to develop and validate alternative metrics that better capture transcription quality for Banglish. These metrics should focus on technical term capture—whether the system successfully identifies and transcribes the keywords and concepts central to lecture content—while appropriately handling transliteration variance. Validation requires demonstrating that the proposed metrics correlate with human judgments of transcription utility.

1.5.2 Secondary Objectives

Beyond the primary objectives, several secondary goals extend the research in valuable directions.

Visual Bias Investigation. We investigate whether injecting visually-extracted keywords directly into the ASR decoding process improves or harms transcription quality. This technique, conceptually appealing as a way to “hint” the ASR toward correct terms, may backfire by causing hallucination of keyword mentions. Systematic experimentation across multiple configurations reveals the conditions under which visual bias helps, harms, or has negligible effect.

Dual-ASR Fusion Exploration. We explore approaches that combine outputs from multiple ASR systems—specifically, English-focused Whisper and Bengali-focused BanglaASR. This exploration includes developing transliteration methods to convert Bengali Unicode output to Romanized form, enabling comparison and fusion with Whisper output. While initial results suggest limited benefit from dual-ASR approaches in our context, the exploration contributes methodological insights for future work.

Anti-Hallucination Mechanism Development. We develop post-processing techniques to detect and remove ASR hallucinations—particularly repetitive pat-

terns and stock phrases—that degrade transcript quality. These mechanisms must balance aggressive hallucination removal against the risk of inadvertently removing legitimate content.

Lecture Note Generation. We implement summarization capabilities that transform raw transcripts and visual information into structured, readable lecture notes. These notes should organize content logically, highlight key concepts, and present information in a format that supports efficient student review. While summarization quality is difficult to evaluate quantitatively, we aim for outputs that students would find genuinely useful.

1.6 Methodology in Brief

This research adopts an experimental methodology that combines iterative system development with rigorous empirical evaluation. Rather than implementing a single predetermined approach and evaluating it, we follow a cycle of hypothesis formation, implementation, evaluation, and refinement that allows us to learn from both successes and failures. This section provides a high-level overview of our methodology; Chapter 4 presents the detailed technical approach.

1.6.1 Phase 1: System Architecture Design

The research began with careful architectural planning. We designed a modular pipeline that processes lecture videos through parallel audio and visual tracks, with multiple points where cross-modal information can influence processing. This architecture allows us to experiment with different fusion strategies without rebuilding the entire system.

The audio track employs OpenAI’s Whisper model (specifically the large-v3-turbo variant) as the primary ASR engine, chosen for its strong multilingual capabilities and demonstrated performance on accented English. We supplement this with BanglaASR, a Wav2Vec2-based model fine-tuned for Bengali, to explore dual-ASR fusion possibilities. Both models operate on audio extracted from input videos using FFmpeg.

The visual track uses Qwen2.5-VL-7B, a state-of-the-art vision-language model, to extract text and keywords from video frames sampled at configurable intervals. This model can interpret complex visual scenes, recognize both printed and handwritten text, and categorize content into keywords, definitions, code snippets, and diagrams. For final summarization, we employ Qwen2.5-7B-Instruct, a large language model capable of synthesizing information from transcripts and visual context into coherent lecture notes. The entire pipeline operates on NVIDIA RTX 3090 hardware with 24GB VRAM, using a model registry that manages GPU memory by loading and unloading models as needed.

1.6.2 Phase 2: Data Collection and Ground Truth Creation

Meaningful evaluation requires ground truth data against which system outputs can be compared. We collected nine real classroom lecture recordings from Bangladeshi educational content, covering three distinct technical domains: Python programming (including variables, input/output, conditionals, loops, and data structures),

Digital Logic Design (logic gates and universal gates), and Database Management Systems (introduction and SQL queries).

For each recording, we manually transcribed the complete lecture content in Romanized Banglish, following consistent conventions while preserving the code-mixed nature of the original speech. This transcription effort produced approximately 73,000 characters of ground truth text across the dataset. The transcriptions capture not just words but the rhythm and structure of actual classroom instruction, including false starts, repetitions, and the natural disfluencies of spontaneous speech.

1.6.3 Phase 3: Experimentation and Iteration

With the pipeline architecture established and ground truth available, we conducted extensive experiments to evaluate various approaches. Visual bias injection was tested at multiple boost intensities, revealing that naive approaches often harm rather than help transcription quality. Temporal alignment strategies attempted to localize visual influence to appropriate time windows, with mixed results. Dual-ASR fusion explored whether combining Whisper and BanglaASR outputs improves coverage, finding limited benefit in our context. Anti-hallucination post-processing was developed and refined to remove repetitive patterns without damaging legitimate content.

Each experiment followed a consistent protocol: modify a specific system component, process all videos in the dataset, compute evaluation metrics against ground truth, and analyze results to inform the next iteration. This disciplined approach ensures that our conclusions rest on systematic evidence rather than anecdotal observation.

1.6.4 Phase 4: Evaluation and Analysis

Evaluation employs the term-based metrics we developed specifically for Banglish technical content. Term Recall measures what fraction of ground truth technical terms appear in the system output, using fuzzy matching to accommodate Romanization variants. Term Precision measures what fraction of technical terms in the system output actually correspond to ground truth terms. Term F1 provides the harmonic mean of precision and recall, offering a balanced summary metric. Fuzzy Similarity provides a character-level similarity score that captures overall transcript quality beyond just technical terms.

We deliberately exclude Word Error Rate from our primary evaluation, having demonstrated that it systematically penalizes valid Romanization variants. This methodological choice reflects our broader commitment to evaluation metrics that capture practical utility rather than artificial precision.

1.7 Scope and Limitations

Every research project operates within boundaries—some chosen deliberately to focus effort, others imposed by constraints of time, resources, or circumstance. Clearly articulating these boundaries helps readers understand what this thesis does and does not claim to accomplish.

1.7.1 Scope of This Research

This research focuses specifically on pre-recorded classroom lecture videos featuring technical content delivered in Banglish. We process complete video files rather than streaming content; real-time transcription, while an interesting extension, falls outside our current scope. The lectures we address feature whiteboard-based instruction, where instructors write and draw while explaining concepts verbally. This focus on whiteboard content enables our visual extraction approaches but limits applicability to purely verbal lectures or those using only projected slides.

Our dataset spans three technical domains: Python programming, Digital Logic Design, and Database Management Systems. These domains were chosen to represent the kinds of computer science and engineering content commonly taught in Bangladeshi universities and coaching centers. The Python lectures cover foundational programming concepts including variables, input/output operations, conditional statements, loop constructs, and data structures like lists and tuples. The Digital Logic Design lectures address basic logic gates (AND, OR, NOT) and universal gates (NAND, NOR). The Database lectures introduce DBMS concepts and SQL query syntax. This domain coverage, while not exhaustive, provides reasonable diversity for evaluating system generalization.

The nine classroom recordings in our dataset represent real-world instructional content with authentic acoustic conditions. These are not studio recordings with professional audio equipment; they are the kinds of recordings that students actually encounter when learning online. This realism ensures that our results reflect genuine system performance rather than best-case scenarios.

1.7.2 Limitations and Constraints

Several limitations constrain the claims we can make and the conclusions we can draw.

Dataset Size. Nine videos, while sufficient for demonstrating methodological approaches and establishing baseline performance, represent a limited sample. We cannot claim with certainty that results would generalize to all Banglish lectures, all technical domains, or all instructional styles. A larger dataset would strengthen confidence in our findings, and we acknowledge this as a limitation that future work should address.

Instructional Style Homogeneity. The recordings in our dataset share certain stylistic characteristics: they feature individual instructors speaking to camera while writing on whiteboards. We have not tested lectures with multiple speakers, panel discussions, student interaction, or dramatically different pedagogical approaches. Our system may perform differently on such varied content.

Hardware Requirements. The full system requires substantial computational resources—specifically, an NVIDIA GPU with at least 24GB of VRAM to run the vision-language model and language model simultaneously, or the ability to swap models in and out of memory sequentially. This requirement limits accessibility for researchers or practitioners with more constrained hardware. While we implement memory management strategies to reduce peak VRAM usage, the computational cost remains significant.

Visual Dependency. Our multimodal approach relies heavily on whiteboard content for visual grounding. Lectures that use only verbal instruction, or that rely

on projected slides with minimal text, may not benefit from our visual extraction pipeline. The system remains functional in these cases—it simply falls back to audio-only processing—but the multimodal advantages we claim require visual content that provides meaningful ground truth.

Language Scope. We address only Bengali-English code-mixing. While similar code-mixing phenomena occur in other language pairs (Hindi-English “Hinglish,” Spanish-English “Spanglish,” and many others), we cannot claim that our approaches transfer directly. Each code-mixing context has unique linguistic characteristics that may require adapted methods.

Evaluation Metric Validation. While we propose term-based metrics as appropriate for Banglish evaluation, these metrics have not been validated against human judgments at scale. We believe they capture practical utility better than WER, but systematic human evaluation studies would strengthen this claim. Such studies fall outside the scope of this thesis but represent important future work.

1.8 Research Contributions

This thesis contributes to the fields of speech recognition, multimodal learning, and low-resource language processing through technical innovations, dataset creation, and empirical findings. We organize these contributions into three categories.

1.8.1 Technical Contributions

Multimodal Pipeline Architecture. We design and implement a complete, functional system for processing Banglish lecture videos. This pipeline integrates multiple state-of-the-art models—Whisper for ASR, Qwen2.5-VL for visual extraction, Qwen2.5 for summarization—into a coherent workflow that transforms raw video input into structured lecture notes. The modular architecture allows components to be upgraded or replaced as technology advances, and the implementation handles practical concerns like GPU memory management that often receive insufficient attention in research prototypes.

Term-Based Evaluation Framework. We develop and validate evaluation metrics specifically designed for Romanized code-mixed languages. Unlike traditional Word Error Rate, our Term Recall, Term Precision, and Term F1 metrics focus on technical keyword capture while accommodating transliteration variance. This evaluation framework can be applied beyond our specific system, providing methodological tools for other researchers working on code-mixed ASR.

Cross-Modal Verification Techniques. We develop approaches for using visual ground truth to verify and potentially correct ASR outputs. These techniques include keyword extraction from whiteboard frames, fuzzy matching between visual and audio terms, and confidence-weighted correction mechanisms. While our experiments reveal that naive visual bias injection can harm performance, the verification approaches provide value for error detection and quality assessment.

Anti-Hallucination Processing. We implement and evaluate methods for detecting and removing ASR hallucinations—particularly repetitive patterns that can severely degrade transcript quality. These post-processing techniques improve output reliability without requiring modifications to the underlying ASR model, making them broadly applicable to other ASR systems and contexts.

1.8.2 Dataset Contribution

BanglaASR Benchmark Dataset. We create and release a benchmark dataset of nine Banglish technical lecture recordings with manually transcribed ground truth. This dataset covers three technical domains (Python, DLD, DBMS), encompasses approximately 73,000 characters of transcribed content, and represents real classroom acoustic conditions. The dataset enables rigorous evaluation of ASR systems on Banglish technical content and provides a foundation for future research in this understudied area. To our knowledge, this represents the first publicly available benchmark for Banglish technical lecture transcription.

1.8.3 Empirical Contributions

Visual Bias Analysis. Through systematic experimentation, we demonstrate that naive visual bias injection—boosting the probability of visually-extracted keywords during ASR decoding—can harm rather than help transcription quality. This finding, while perhaps counterintuitive, provides important guidance for researchers considering similar approaches. We characterize the conditions under which visual bias helps versus harms, contributing nuanced understanding of multimodal fusion challenges.

Metric Appropriateness Demonstration. We provide empirical evidence that traditional Word Error Rate is inappropriate for evaluating Romanized Banglish transcription. By showing specific examples where correct transcriptions receive high WER scores due to valid Romanization variants, we justify the need for alternative metrics and validate our term-based approach.

Performance Baselines. Our experiments establish quantitative baselines for Banglish technical lecture transcription: 73.9% Term F1, 83.6% Term Precision, and 66.5% Term Recall. These figures provide reference points for future research, enabling meaningful comparison as techniques improve. The baselines represent honest assessment of current capabilities, neither inflated through cherry-picked examples nor deflated through pessimistic evaluation.

Chapter 2

Literature Review

2.1 Introduction

This chapter provides a comprehensive survey and critical analysis of the research landscape that informs and contextualizes our work on multimodal Banglish lecture transcription. We examine five interconnected areas in depth: automatic speech recognition, with particular attention to its evolution toward multilingual and low-resource settings; code-mixed language processing, focusing on the South Asian context where Banglish emerges; multimodal learning and vision-language models that enable cross-modal information fusion; lecture video processing systems that represent prior attempts to automate educational content transformation; and evaluation methodologies for ASR and code-mixed languages. For each area, we identify key contributions, critically analyze methodological approaches, compare competing paradigms, and highlight gaps that our research addresses.

Our analysis reveals a fundamental tension in the literature: while individual components of our problem—ASR, multimodal fusion, code-mixing—have received substantial attention, their intersection remains largely unexplored. This chapter not only surveys existing work but critically examines why these research threads have remained disconnected and how our approach bridges them.

2.2 Automatic Speech Recognition

2.2.1 Historical Evolution and Deep Learning Revolution

Automatic speech recognition has undergone a remarkable transformation over the past decade, evolving from systems requiring careful acoustic modeling and language-specific engineering to end-to-end neural approaches that learn directly from data. Understanding this evolution is crucial for appreciating both the capabilities and limitations of current systems when applied to challenging domains like Banglish technical lectures.

The foundations of modern ASR were established through decades of research on Hidden Markov Models (HMMs) combined with Gaussian Mixture Models (GMMs), which dominated the field from the 1980s through the early 2010s **Rabiner1989, Juang1991**. These systems required extensive feature engineering, including Mel-frequency cepstral coefficients (MFCCs), and relied on separate acoustic, pronunciation, and language models trained independently. The modular architecture of

ferred interpretability—errors could be traced to specific components—but required substantial expertise for each language and domain. Critically for our work, this modularity meant that supporting a new language required building all components from scratch: acoustic models, pronunciation dictionaries, and language models. For under-resourced languages like Bengali, this created insurmountable barriers.

The deep learning revolution fundamentally altered this landscape but introduced new challenges. Hinton et al. **Hinton2012** demonstrated that deep neural networks could replace GMMs in the acoustic modeling component, achieving 20-30% relative error reduction on benchmark tasks. This breakthrough catalyzed rapid progress: recurrent neural networks (RNNs) with Long Short-Term Memory (LSTM) cells **Hochreiter1997** enabled modeling of temporal dependencies in speech that HMMs struggled to capture, while the Connectionist Temporal Classification (CTC) loss function **Graves2006** allowed training without pre-aligned transcriptions—a critical advance that reduced annotation requirements.

However, a careful analysis of CTC-based systems reveals limitations relevant to our domain. CTC assumes conditional independence between output tokens given the input, which works well for acoustic-to-phoneme mapping but poorly captures linguistic dependencies. For code-mixed speech, where the language context strongly influences word probabilities, this independence assumption is particularly problematic. A Bengali word following an English technical term has different probability than the same word in a purely Bengali context—a dependency CTC cannot model. The introduction of attention mechanisms **Bahdanau2015** and the Transformer architecture **Vaswani2017** marked another paradigm shift that addressed some CTC limitations. Chan et al. **Chan2016** proposed Listen, Attend and Spell (LAS), an end-to-end model that jointly learned acoustic and language modeling through attention. This approach eliminated the need for separate pronunciation dictionaries and language models, simplifying the ASR pipeline while achieving competitive performance. The attention mechanism allowed the decoder to focus on relevant portions of the acoustic input when generating each output token, implicitly learning alignment without explicit supervision.

Critical Analysis: While LAS and similar attention-based models represent significant advances, their data requirements pose challenges for low-resource scenarios. The original LAS paper reported training on 12,500 hours of English speech—a dataset size unavailable for most languages. Furthermore, the end-to-end nature that simplifies deployment also complicates adaptation: there is no separate language model to swap for domain-specific vocabulary, no pronunciation dictionary to extend with new terms. For Banglish technical content, where domain vocabulary is crucial and training data is scarce, these limitations are significant.

2.2.2 Self-Supervised Learning and Foundation Models

A pivotal development in recent ASR research has been the application of self-supervised learning to speech, fundamentally changing the resource requirements for building capable systems. This paradigm shift has profound implications for low-resource languages, though as we analyze below, significant gaps remain for code-mixed scenarios.

Wav2Vec 2.0 **Baevski2020** demonstrated that pre-training on unlabeled audio through contrastive learning could produce representations that dramatically re-

duced the labeled data requirements for downstream ASR tasks. The key insight was that abundant unlabeled audio—available even for low-resource languages through radio broadcasts, YouTube, and other sources—could provide the acoustic knowledge that previously required expensive manual transcription. Fine-tuning on just 10 minutes of labeled data achieved results competitive with systems trained on 100 hours, a 600x reduction in annotation requirements.

Methodological Analysis: The Wav2Vec 2.0 architecture combines a CNN-based feature encoder with a Transformer-based context network. During pre-training, the model learns to identify the correct quantized representation of masked audio segments from a set of distractors—a contrastive objective analogous to masked language modeling in NLP. This approach implicitly learns phonetic categories and acoustic invariances without explicit phoneme labels. However, our analysis reveals that the learned representations remain language-specific: a model pre-trained on English audio captures English phonetic distinctions but may miss phonemes unique to Bengali (e.g., retroflex consonants, aspirated stops). Cross-lingual transfer helps but does not fully bridge this gap.

HuBERT **Hsu2021** extended self-supervised speech representation learning through an iterative clustering approach, achieving state-of-the-art results on several benchmarks. Unlike Wav2Vec 2.0’s contrastive objective, HuBERT uses a predictive objective: the model learns to predict cluster assignments for masked regions, where clusters are derived from previous model iterations or acoustic features. This iterative refinement progressively discovers more linguistically meaningful units. On the LibriSpeech benchmark, HuBERT achieved 1.4% word error rate with fine-tuning on 960 hours of labeled data—approaching human performance.

Critical Comparison: Comparing Wav2Vec 2.0 and HuBERT reveals trade-offs relevant to our application. Wav2Vec 2.0’s contrastive learning is more stable during training but may learn representations tied to dataset-specific acoustic conditions. HuBERT’s iterative refinement captures more abstract linguistic structure but requires careful hyperparameter tuning and multiple training passes. For code-mixed content, neither approach explicitly addresses the challenge of multiple language varieties in the same utterance—both assume relatively homogeneous training data. The XLSR (Cross-Lingual Speech Representations) line of work **Conneau2021** extended self-supervised pre-training to 53 languages, demonstrating that a single model could capture phonetic knowledge transferable across language families. These self-supervised models have been adapted for numerous languages, including Bengali, through fine-tuning on relatively small labeled datasets. The BanglaASR model we employ in our experiments derives from this lineage: Wav2Vec 2.0 pre-trained on multilingual data and fine-tuned on Bengali speech.

Limitations for Code-Mixing: Despite these advances, our analysis identifies a critical gap: self-supervised models are pre-trained on monolingual data and assume language consistency within utterances. When an utterance contains systematic code-mixing—as in Banglish—the model encounters acoustic patterns that violate its training assumptions. An English word pronounced with Bengali phonology falls outside both the English and Bengali acoustic spaces the model learned. This mismatch partly explains why both Whisper and BanglaASR struggle with Banglish: neither was trained on code-mixed data, and the acoustic realization of code-mixed speech differs from either source language.

2.2.3 Whisper: Multilingual ASR at Scale

OpenAI’s Whisper **Radford2023** represents a significant milestone in multilingual ASR and merits detailed analysis given its central role in our pipeline. Unlike previous multilingual models that required language-specific fine-tuning, Whisper demonstrates robust performance across 99 languages from a single model checkpoint, fundamentally changing expectations for multilingual ASR.

Architecture and Training: Whisper employs a standard encoder-decoder Transformer architecture, processing log-Mel spectrograms through a CNN front-end followed by Transformer encoder blocks. The decoder generates text tokens autoregressively, conditioned on the encoded audio and previous tokens. Special tokens indicate task (transcription vs. translation), language, and timestamps. Trained on approximately 680,000 hours of labeled audio data collected from the internet—roughly 100x the data used for previous large-scale ASR models—Whisper achieved unprecedented robustness across domains and acoustic conditions.

Training Methodology Analysis: Whisper’s training methodology emphasizes robustness over benchmark optimization, a design philosophy worth examining critically. By training on diverse, “in-the-wild” audio including varying acoustic conditions, background noise, and speaker characteristics, the model achieves generalization that surpasses systems optimized for clean benchmark conditions. The weak supervision approach—using automatically-aligned audio-text pairs from the internet rather than carefully transcribed datasets—introduces label noise but provides scale and diversity impossible with manual transcription.

This design philosophy aligns well with our target domain of classroom recordings, which feature realistic acoustic challenges. However, the training data composition reveals potential biases: English comprises approximately 65% of training data, while Bengali represents less than 0.5%. This imbalance manifests in Whisper’s behavior on Banglish: the model strongly prefers English transcription, interpreting Bengali words through an English phonetic lens.

Language Handling Behavior: However, Whisper’s multilingual capabilities have significant limitations for code-mixed content that our analysis reveals. The model tends to transcribe in a single language, determined either automatically through a language identification module or through explicit language specification via special tokens. When processing Banglish, Whisper typically defaults to English transcription, handling English portions adequately but producing phonetically-motivated English approximations for Bengali words **Peng2023**.

Our experimental analysis (detailed in Chapter 5) quantifies this behavior: when processing our Banglish lecture dataset with English language specification, Whisper transcribes English technical terms with 85% accuracy but reduces Bengali words to phonetic nonsense (“korbo” becomes “curb oh,” “bujhte” becomes “boost a”). With Bengali language specification, the reverse occurs: Bengali words are transcribed in Bengali script while English terms are phonetically approximated in Bengali characters. Neither mode handles the code-mixed reality of Banglish instruction.

Comparison with Alternatives: Comparing Whisper to alternatives reveals trade-offs in our application context:

- **Google Speech-to-Text** offers Bengali language support but, like Whisper, assumes monolingual content. Commercial pricing limits research experimentation.

- **Azure Speech Services** provides custom model training but requires substantial labeled data for adaptation—precisely the resource we lack for Banglish.
- **Wav2Vec 2.0 fine-tuned models** offer language-specific optimization but require training infrastructure and data we aim to avoid.
- **Whisper** provides the best balance of availability, robustness to acoustic conditions, and baseline multilingual capability, making it our primary ASR engine despite code-mixing limitations.

This analysis motivates our multimodal approach: rather than attempting to train a code-mixed ASR model (infeasible without large-scale Banglish training data), we leverage visual information to verify and correct Whisper’s outputs, accepting its English-biased transcription as a starting point for refinement.

2.2.4 Bengali and South Asian Language ASR

Research on Bengali ASR has accelerated in recent years, though resources remain limited compared to high-resource languages. A comprehensive analysis of existing work reveals both progress and persistent gaps that frame our contribution.

The Bengali.AI Speech Recognition Challenge **BengaliAI2023** released a substantial dataset of read speech, catalyzing development of Bengali-specific models. This dataset comprises approximately 1,200 hours of speech from 5,000 speakers, covering diverse dialects and demographic groups. The challenge attracted significant participation, with winning systems achieving word error rates below 10% on the evaluation set. However, critical analysis reveals limitations of this benchmark for our application:

- **Read vs. Spontaneous Speech:** The dataset consists of read speech—speakers reading predefined sentences—which differs acoustically and linguistically from spontaneous classroom instruction. Read speech has cleaner articulation, fewer disfluencies, and predictable prosody. Spontaneous speech, as in our lectures, includes false starts, self-corrections, filler words, and variable speaking rates that challenge ASR systems.
- **Monolingual Content:** The dataset is purely Bengali, excluding code-mixed content. Systems optimized on this data have no exposure to English technical terms embedded in Bengali sentences.
- **Script Output:** The task requires Bengali Unicode output, while our application needs Romanized Banglish for accessibility and practical use.

BanglaASR, built on Wav2Vec 2.0 fine-tuned on Bengali data, represents one of the more capable open-source Bengali ASR systems **BanglaASR2023**. Our experimental evaluation (Chapter 5) reveals both its strengths and limitations: BanglaASR handles Bengali phonology well, correctly recognizing retroflex consonants and vowel distinctions that Whisper misses. However, English technical terms become unintelligible Bengali approximations (“object” becomes meaningless Bengali syllables), and the Bengali Unicode output requires transliteration for comparison with Romanized ground truth—a lossy process that introduces additional errors.

System	Bengali	English	Code-Mixed	Output
Whisper (en)	Poor	Good	Poor	Roman
Whisper (bn)	Moderate	Poor	Poor	Bengali
BanglaASR	Good	Poor	Poor	Bengali
Google STT	Moderate	N/A	N/A	Bengali

Table 2.1: Comparative capabilities of ASR systems on Banglish content

Comparative Analysis of Bengali ASR Approaches:

Ahmed et al. **Ahmed2022** noted that existing Bengali ASR systems struggle with English technical terms, while systems optimized for English fail on Bengali morphology. Their analysis of error patterns—which we extend in our work—identified three primary failure modes: (1) phoneme substitution, where sounds outside the model’s training language are mapped to nearest equivalents; (2) segmentation errors, where word boundaries are misidentified in code-mixed contexts; and (3) script confusion, where the model attempts to transcribe foreign words in the wrong script. This gap motivates our multimodal approach to leverage visual information for terminology verification.

South Asian language ASR more broadly faces similar challenges, and comparing approaches across languages provides methodological insights. Research on Hindi-English code-mixing (Hinglish) has explored language identification and switching detection **Sitaram2019**, with some systems achieving over 90% accuracy on language identification at the word level. However, word-level code-mixing with morphological integration—where a single token contains elements from both languages—remains challenging. The morphological richness of South Asian languages, combined with productive derivational processes that apply native morphology to borrowed stems, creates tokens that are genuinely bilingual at the sub-word level.

Choudhury et al. **Choudhury2007** analyzed code-mixing patterns in Indian languages, finding that technical domains exhibit particularly high rates of English term insertion, often with native language grammatical markers. Their corpus analysis of social media and educational content revealed that code-mixing rates in technical discussions exceed 40%—meaning nearly half of content involves language alternation. This finding directly informs our work: Banglish technical lectures are not marginally code-mixed but pervasively bilingual, requiring systems designed for this reality rather than treating code-mixing as an edge case.

2.3 Code-Mixed Language Processing

Code-mixing represents one of the most challenging phenomena for natural language processing systems, requiring simultaneous competence in multiple languages and understanding of how they interact. This section provides an in-depth analysis of code-mixing research, with particular attention to the South Asian context where Banglish emerges.

2.3.1 Linguistic Foundations of Code-Mixing

Code-mixing and code-switching represent distinct multilingual phenomena with different implications for natural language processing, and understanding their the-

oretical foundations is essential for principled system design. Code-switching involves alternation between languages at clause or sentence boundaries, maintaining the grammatical integrity of each language within its segment **MyersScotton1993**. Code-mixing, in contrast, involves the integration of elements from two languages within a single utterance, often at the word or morpheme level **Muysken2000**.

Theoretical Framework Analysis: The Matrix Language Frame (MLF) model **MyersScotton1997** provides the dominant theoretical framework for understanding code-mixing patterns. According to this model, one language (the matrix language) provides the grammatical framework—word order, morphological affixes, and function words—while elements from another language (the embedded language) are inserted at designated points. The model predicts constraints on where embedded language elements can appear: they must either be well-formed constituents or must receive matrix language morphology.

In Banglish technical discourse, Bengali typically serves as the matrix language, providing word order, morphological affixes, and function words, while English provides technical vocabulary and some verbs. Our corpus analysis confirms this pattern: Bengali provides sentence structure (SOV word order), case marking, and tense/aspect morphology, while English contributes nouns (particularly technical terms), some verbs, and occasional adjectives. For example, “amra class theke object create korbo” exhibits Bengali matrix structure with English embedded elements (“class,” “object,” “create”) that receive Bengali morphology (“-theke” locative, “korbo” future tense).

Implications for NLP: Understanding these linguistic patterns is crucial for NLP system design. Systems that treat code-mixing as simple language alternation, applying monolingual models sequentially, fail to capture the grammatical integration that characterizes genuine code-mixing. The morphological attachment of Bengali suffixes to English stems (e.g., “compile-korbo” for “will compile”) requires truly bilingual processing that neither monolingual system provides.

Muysken’s Typology: Muysken **Muysken2000** proposed a typology distinguishing three code-mixing patterns: *insertion* (embedding single words or phrases from one language into another’s structure), *alternation* (switching between languages at constituent boundaries), and *congruent lexicalization* (sharing grammatical structure while drawing vocabulary from both languages). Banglish technical discourse exhibits primarily insertion patterns for technical terms but alternation for longer English explanations. Recognizing these patterns helps predict where code-mixing will occur and what forms it will take.

2.3.2 Code-Mixed Language Identification and Processing

Language identification in code-mixed text has been studied extensively, with shared tasks at venues like FIRE and SemEval driving progress **Solorio2014**, **Aguilar2018**. These tasks typically address word-level language tagging, identifying which language each token belongs to. A detailed analysis of approaches and their limitations reveals challenges relevant to our work.

Shared Task Analysis: The FIRE 2013 and subsequent shared tasks on code-mixed language identification established benchmarks for Hindi-English, Bengali-English, and other language pairs. Early systems relied on lexicon-based approaches, classifying words as belonging to the language in whose dictionary they appeared.

However, this approach fails for words present in both languages (“data” is valid English and borrowed into Bengali), named entities, and transliterated words. Machine learning approaches using character n-grams achieved better performance by capturing orthographic patterns distinctive to each language.

The best-performing systems at recent shared tasks achieve 90-95% token-level accuracy on language identification. However, performance degrades significantly for morphologically complex cases where a single token contains elements from both languages. A word like “install-korbo” (“will install”) cannot be cleanly assigned to either language—it is genuinely bilingual at the token level. For ASR evaluation, where word boundaries may be incorrectly identified, this limitation compounds.

Subword Tokenization Analysis: Subword tokenization approaches, particularly Byte-Pair Encoding (BPE) [Sennrich2016](#) and WordPiece [Schuster2012](#), offer partial solutions by segmenting words into smaller units that may align with language boundaries. For example, “install-korbo” might be tokenized as “install,” “-kor,” “bo,” potentially allowing the English stem and Bengali morphology to be processed separately. However, these tokenizers are typically trained on monolingual or parallel corpora rather than code-mixed data, potentially fragmenting code-mixed tokens in linguistically unmotivated ways.

Our analysis of Whisper’s tokenization on Banglish reveals this problem: the tokenizer, trained primarily on English, treats Bengali words as sequences of unfamiliar characters, fragmenting them into short, meaningless pieces. This fragmentation disrupts the model’s language modeling, as it never encountered these token sequences during training. The resulting outputs combine correctly-tokenized English words with garbled Bengali, producing transcriptions that are neither good English nor good Bengali.

Pre-trained Model Adaptation: Recent work has explored training tokenizers and language models specifically on code-mixed data. [Khanuja et al. Khanuja2020](#) released GLUECoS, a benchmark for code-mixed natural language understanding that includes Hindi-English and Spanish-English data across tasks including sentiment analysis, named entity recognition, and question answering. Their experiments demonstrated that multilingual pre-trained models like mBERT [Devlin2019](#) provide reasonable baselines but leave substantial room for improvement on code-mixed tasks. Specifically, mBERT’s performance dropped 15-20% absolute on code-mixed data compared to monolingual equivalents, suggesting that multilingual pre-training does not automatically confer code-mixed competence.

Gap Analysis: Critically for our work, we note that GLUECoS and similar benchmarks address written text rather than speech. The challenges of code-mixed text processing—language identification, tokenization, semantic understanding—are compounded in speech by acoustic variation, pronunciation adaptation, and the absence of visual word boundaries. No equivalent benchmark exists for code-mixed speech in the Banglish variety, motivating our dataset contribution.

2.3.3 Code-Mixed Speech Recognition

ASR for code-mixed speech presents unique challenges beyond those faced in text processing, requiring consideration of acoustic, phonetic, and linguistic factors simultaneously. A systematic analysis of existing approaches reveals the limitations that our multimodal approach addresses.

Acoustic Challenges: The acoustic realization of code-mixed utterances may differ from either component language, with speakers adapting pronunciation based on the surrounding linguistic context. An English word embedded in Bengali speech may receive Bengali phonological treatment, affecting vowel quality, stress patterns, and consonant articulation. For example, an English word like “class” in Banglish may be pronounced with Bengali retroflex /l/, final consonant gemination, or vowel adaptation. These modifications, natural for bilingual speakers, confuse ASR systems trained on native English or Bengali pronunciation.

Conversely, Bengali words in technical contexts may receive English-influenced pronunciation. A Bengali speaker discussing programming might unconsciously shift toward English intonation patterns even when using Bengali words. These prosodic accommodations provide cues to human listeners about language switching but may mislead ASR language identification.

Approach Analysis: Research on code-mixed ASR has explored several approaches with varying success:

- **Language-Switching Models:** Yilmaz et al. **Yilmaz2016** investigated bilingual deep neural networks for Frisian-Dutch code-switching, using language-specific acoustic models with switching mechanisms that select the appropriate model based on detected language. This approach achieved 5-10% relative WER improvement over monolingual baselines. However, it struggles with word-internal code-mixing (common in Banglish) and requires accurate language boundary detection—itself a challenging problem in continuous speech.
- **Joint Multilingual Models:** Li et al. **Li2019** proposed multi-encoder architectures that process audio through parallel language-specific encoders before fusion. This approach shows promise for capturing language-specific acoustic patterns while allowing cross-language information sharing. However, it requires substantial code-mixed training data to learn appropriate fusion strategies—data that is scarce for most language pairs including Bengali-English.
- **Language-Agnostic Approaches:** Some researchers have explored training models without explicit language labels, hoping the model discovers language structure implicitly. Results have been mixed: such models may learn acoustically-grounded representations but lack the linguistic knowledge to produce coherent output in either language.

Critical Gap: For Banglish specifically, we found no published work addressing code-mixed ASR with ground truth evaluation. This absence is striking given the prevalence of Banglish in Bangladeshi education and the millions of students who could benefit from accurate transcription. The gap reflects broader patterns in AI research, where languages spoken by billions receive a fraction of the attention devoted to English. Our dataset creation addresses this gap, providing the first benchmark for Banglish technical lecture ASR.

2.3.4 Banglish: Characteristics and Challenges

Banglish exhibits distinctive characteristics that differentiate it from other code-mixed varieties and require specific consideration in system design. Our analysis

draws on both prior literature and our own corpus observations.

Bali et al. **Bali2014** analyzed social media code-mixing patterns, finding that technical and educational domains show particularly high rates of English insertion. Their corpus of 1,200 Facebook posts revealed code-mixing rates of 36% (by token count) in technical discussions, compared to 18% in casual social contexts. Unlike casual social media Banglish, which may exhibit more varied mixing patterns, educational Banglish shows systematic patterns: technical terms appear in English while explanatory language remains primarily Bengali.

Romanization Complexity: The Romanization of Banglish introduces additional complexity absent from code-mixed varieties with established orthographies. While Hindi-English code-mixing can leverage Devanagari script for Hindi portions—providing unambiguous orthography—Bangladeshi users frequently write Bengali in Roman script, especially in digital contexts. This Romanization follows no standard convention, resulting in high orthographic variability. The same Bengali word may be spelled differently by different writers, or even by the same writer in different contexts.

Our corpus analysis identified three primary sources of Romanization variance:

1. **Vowel Representation:** Bengali distinguishes vowel length and quality in ways not directly represented in Roman orthography. The word for “we” might be Romanized as “amra,” “aamra,” “amraa,” or “amara” depending on writer preference.
2. **Consonant Clusters:** Bengali consonant clusters may be simplified or preserved variably. The word for “with” appears as “shathe,” “sathe,” “shate,” or “saathe.”
3. **Retroflex Sounds:** Bengali distinguishes dental and retroflex consonants, but Roman script lacks standardized representation. Writers may use “t” for both, or use “T” or “th” for retroflex variants inconsistently.

Implications for Evaluation: This orthographic variability has profound implications for NLP systems and evaluation. Traditional approaches that rely on lexicon lookup or exact string matching fail when faced with multiple valid spellings. A system that produces “aamra” when the reference says “amra” is not wrong—both are valid—but would be penalized by exact-match metrics. Fuzzy matching approaches must balance precision against recall, risking either false negatives (failing to match valid variants) or false positives (incorrectly matching unrelated words). Our term-based evaluation metrics explicitly address this challenge through similarity thresholds rather than exact matching, accepting predictions within 85% character similarity as matches.

Technical Vocabulary Patterns: Our analysis of lecture transcripts reveals consistent patterns in how technical vocabulary is integrated:

- **Nouns:** Technical nouns almost universally appear in English (“variable,” “function,” “database”), often with Bengali case markers attached.
- **Verbs:** Technical verbs show mixed patterns. Some appear as English stems with Bengali light verb (“create kora,” “define kora”), while others are nativized (“compile korbo” paralleling Bengali verb morphology).

- **Adjectives:** Technical adjectives frequently remain English (“primary key,” “object-oriented”), though descriptive adjectives may be Bengali.

These patterns inform our visual bias approach: whiteboard content, being primarily technical vocabulary, predominantly contains the English terms that Whisper handles well. Visual extraction thus provides ground truth for precisely the tokens most likely to be correct in Whisper output—a fortuitous alignment that our system exploits.

2.4 Multimodal Learning and Vision-Language Models

Multimodal learning represents one of the most dynamic areas of AI research, with vision-language models achieving capabilities that seemed impossible just a few years ago. This section provides comprehensive analysis of multimodal approaches relevant to our work, examining both foundations and recent advances with attention to their applicability and limitations for lecture video processing.

2.4.1 Foundations of Multimodal Learning

Multimodal learning seeks to build models that can process and relate information from multiple modalities such as text, images, audio, and video **Baltrušaitis2019**. The fundamental premise is that different modalities provide complementary information: what is ambiguous in one modality may be clear in another. For lecture video processing, the spoken audio and visual whiteboard content represent complementary channels that together convey the full instructional message.

Taxonomic Analysis: Baltrušaitis et al. **Baltrušaitis2019** provide a comprehensive taxonomy of multimodal challenges: *representation* (how to represent and summarize multimodal data), *translation* (how to map from one modality to another), *alignment* (identifying relationships between elements from different modalities), *fusion* (integrating information from multiple modalities for prediction), and *co-learning* (transferring knowledge between modalities). Our work primarily addresses alignment (connecting visual keywords to audio mentions) and fusion (combining visual and audio evidence for improved transcription).

Fusion Strategy Analysis: Early multimodal approaches typically involved modality-specific encoders followed by fusion mechanisms. Understanding these strategies illuminates choices in our pipeline:

- **Early Fusion:** Combines raw or lightly processed features from different modalities before main processing. Allows learning of low-level cross-modal correlations but requires modalities to be temporally aligned and may be dominated by the higher-dimensional modality.
- **Late Fusion:** Processes each modality independently, combining only final predictions or high-level features. Allows modality-specific optimization but may miss cross-modal interactions that require joint reasoning.

- **Hybrid Fusion:** Combines elements of early and late fusion, with multiple fusion points. Attention-based fusion **Xu2015** allows the model to dynamically weight contributions from different modalities, learning where to attend based on task demands. Tensor fusion **Zadeh2017** captures multiplicative interactions between modality representations, modeling second-order effects.

Our pipeline employs a form of late fusion: audio and visual channels are processed independently (ASR and visual extraction), with fusion occurring at the keyword verification stage. This design choice reflects practical constraints (pre-trained models expect specific input formats) and allows interpretable error analysis (we can identify whether errors originate from audio or visual processing).

Temporal Alignment Challenges: Lecture video processing presents particular alignment challenges. Visual content (whiteboard writing) persists over time, while audio content is inherently sequential. A term written on the whiteboard at minute 5 may be discussed verbally at minutes 5, 10, and 15. Determining which visual content is relevant to which audio segment requires modeling temporal dynamics that simple frame-level processing misses.

Prior work on video understanding has addressed temporal modeling through various mechanisms: 3D CNNs that convolve over time, temporal attention that learns to weight different time steps, and hierarchical approaches that model events at multiple timescales. However, the specific pattern in lecture videos—persistent visual content with intermittent verbal reference—differs from typical video understanding scenarios (action recognition, video captioning) where visual content changes more dynamically.

2.4.2 Vision-Language Pre-training

The remarkable success of large-scale pre-training in NLP, exemplified by BERT **Devlin2019** and GPT **Brown2020**, inspired similar approaches for vision-language modeling. Understanding this evolution contextualizes the capabilities of models we employ.

CLIP Analysis: CLIP **Radford2021** demonstrated that contrastive learning on 400 million image-text pairs from the internet could produce powerful visual representations aligned with natural language. The key insight was that internet-scale image-text pairs, though noisy, provided sufficient signal for learning visual concepts that transfer across tasks. This approach enabled zero-shot image classification, where the model classifies images into arbitrary categories specified through text prompts.

CLIP’s contrastive objective trains the model to associate matching image-text pairs while separating non-matching pairs. The resulting representation space aligns visual and textual concepts: the embedding of an image of a cat lies close to the embedding of the phrase “a photo of a cat.” This alignment enables novel applications impossible with vision-only or language-only models.

Limitations for Our Domain: However, CLIP’s training data and objectives create limitations for lecture video processing:

- **Natural Image Bias:** CLIP was trained on natural images with descriptive captions, not document images or whiteboard photographs. Its visual encoder may struggle with the text-heavy, diagram-rich content typical of lecture whiteboards.

- **Caption vs. OCR:** CLIP learns to describe images, not read them. The textual concepts it captures are about image content, not the specific text appearing in images.
- **Static Images:** CLIP processes single images, not video. Temporal reasoning about content evolution requires additional mechanisms.

BLIP Series: Subsequent work extended vision-language pre-training to more complex tasks. BLIP **Li2022BLIP** introduced a bootstrapping approach that improved training data quality through caption filtering and generation. Recognizing that web-scraped image-text pairs are noisy, BLIP uses a captioning model to generate synthetic captions and a filter to remove low-quality pairs. This data cleaning improved downstream performance across tasks.

BLIP-2 **Li2023BLIP2** proposed a query-based architecture that efficiently bridges frozen image encoders and language models through a lightweight Querying Transformer (Q-Former). Rather than fine-tuning billion-parameter vision and language models end-to-end, BLIP-2 trains only the Q-Former, dramatically reducing computational requirements while achieving competitive performance. The Q-Former learns to extract visual features relevant to language model processing, effectively translating between modalities.

2.4.3 Large Vision-Language Models

The emergence of large language models (LLMs) with instruction-following capabilities **Ouyang2022** catalyzed development of vision-language models that could engage in complex visual reasoning and conversation. This section analyzes key models and their relevance to our application.

LLaVA: LLaVA **Liu2023LLaVA** demonstrated that connecting a vision encoder to an instruction-tuned LLM through a simple projection layer, followed by fine-tuning on visual instruction data, produced capable visual assistants. The key insight was that the language model already possessed reasoning capabilities; it simply needed visual input in a compatible format. LLaVA achieved this by projecting CLIP visual embeddings into the language model’s embedding space, allowing visual information to participate in attention alongside text.

LLaVA’s visual instruction tuning data, generated by prompting GPT-4 with image captions and bounding boxes, taught the model to engage in detailed visual conversations. The resulting system could answer complex questions about images, describe fine details, and reason about spatial relationships. However, LLaVA’s reliance on CLIP visual features inherits CLIP’s limitations for document and text-heavy images.

Qwen-VL Series: Qwen-VL **Bai2023QwenVL** and its successor Qwen2-VL **Wang2024Qwen2VL** represent state-of-the-art open-source vision-language models with capabilities particularly relevant to our application. Unlike models built on CLIP, Qwen-VL uses a vision encoder trained from scratch on diverse data including documents, charts, and screenshots. This design choice enables superior performance on text-heavy images.

Key capabilities relevant to our work include:

- **Arbitrary Resolution:** Qwen2-VL supports images of any aspect ratio and

resolution, important for whiteboard photographs that may be wide, tall, or vary across frames.

- **Multi-Image Context:** The model can process multiple images in a single context, enabling comparison across lecture frames or aggregation of visual information.
- **OCR and Document Understanding:** Qwen2-VL achieves strong performance on document understanding benchmarks, suggesting capability for reading whiteboard text.
- **Grounding:** The model can identify regions of images corresponding to textual descriptions, potentially useful for locating specific content on whiteboards.

The Qwen2.5-VL-7B model we employ in our pipeline achieves competitive performance with much larger proprietary models on visual question answering and text extraction benchmarks. Our experiments (Chapter 5) validate its capability for whiteboard keyword extraction, though we observe failure modes including hallucination of text not present in images and occasional character-level errors in extracted text.

Comparative Analysis:

Model	OCR Quality	Open Source	Context Length	VRAM (7B)
GPT-4V	Excellent	No	128K	N/A
Claude 3	Excellent	No	200K	N/A
LLaVA 1.5	Moderate	Yes	4K	14GB
Qwen2.5-VL	Good	Yes	32K	16GB

Table 2.2: Comparison of vision-language models for OCR-heavy applications

2.4.4 Cross-Modal Verification and Fusion

Several research threads explore using information from one modality to verify or improve processing in another. Our approach draws on these precedents while addressing domain-specific challenges.

Audio-Visual Speech Recognition: In the speech domain, audio-visual speech recognition (AVSR) leverages lip movements to improve ASR accuracy, particularly in noisy conditions **Afouras2018**. The visual modality provides information redundant with but independent of the audio signal, enabling error correction when one modality is degraded. Afouras et al. demonstrated that AVSR could recover from 50% of acoustic errors by incorporating visual lip reading, particularly for consonants whose acoustic realizations are similar but lip shapes are distinct.

Critical Analysis: However, AVSR differs fundamentally from our approach in several ways:

- **Information Type:** AVSR extracts phonetic information (lip shapes correspond to phonemes), while we extract semantic information (whiteboard content provides keywords, not pronunciation).

- **Temporal Alignment:** Lip movements are tightly synchronized with speech, while whiteboard content persists across extended time periods.
- **Coverage:** Lip reading potentially corrects any word, while whiteboard content only provides ground truth for written terms.

These differences mean AVSR techniques do not directly transfer, but the principle—using visual redundancy to correct audio errors—inspires our approach.

How2 Dataset and Multimodal Learning: Sanabria et al. **Sanabria2018** introduced the How2 dataset for multimodal learning from instructional videos, though this dataset features English-only content. Their experiments demonstrated that jointly modeling audio, video, and text improved summarization quality compared to single-modality approaches. The improvement was particularly notable for content where visual demonstrations complemented verbal explanations—precisely the pattern we observe in classroom instruction.

Visual Bias in ASR: The specific approach of biasing ASR with visually-extracted keywords has received limited attention in the literature. Most multimodal ASR work focuses on audio-visual speech recognition using lip reading, which provides phonetic information complementary to audio. Our approach of extracting semantic keywords from whiteboard content and using them to verify or correct ASR output represents a less-explored direction with distinct challenges.

Our Contribution to This Literature: We identify and experimentally characterize a key risk in visual bias approaches: hallucination. When ASR is biased toward visually-extracted keywords, it may over-apply these keywords, inserting them at positions where they were not actually spoken. Our experiments quantify this risk and explore mitigation strategies, contributing empirical understanding to guide future work in visual-semantic ASR enhancement.

2.5 Lecture Video Processing and Educational Technology

Educational technology represents a massive industry with significant academic research investment, yet certain populations remain underserved. This section analyzes existing work on lecture video processing, identifying both achievements and gaps that motivate our contribution.

2.5.1 Automatic Lecture Transcription

Automatic transcription of lecture videos has been studied extensively, driven by accessibility requirements and the desire to make educational content searchable. The evolution of approaches reflects broader ASR advances while revealing domain-specific challenges.

Early Systems: Glass et al. **Glass2007** developed early systems for MIT OpenCourseWare that combined ASR with slide alignment and indexing. These systems demonstrated practical utility despite imperfect transcription accuracy, enabling keyword search and navigation within lecture videos. Their key insight was that

search functionality remains useful even with transcription errors, as long as keywords are occasionally correct. This observation informs our evaluation philosophy: we prioritize technical term recall over overall word accuracy.

Domain Adaptation: More recent work has leveraged improvements in ASR to achieve higher transcription quality. Zhang et al. **Zhang2021** explored using Transformer-based ASR models for lecture transcription, finding that domain adaptation through fine-tuning on academic speech improved performance by 15-25% relative WER reduction. Their analysis identified three primary error sources in lecture transcription:

- **Technical Vocabulary:** Domain-specific terms absent from general ASR training data
- **Disfluencies:** False starts, repetitions, and self-corrections common in spontaneous speech
- **Acoustic Variability:** Varying distances from microphone, background noise, room acoustics

All three challenges appear in our Banglish dataset, compounded by code-mixing. However, their work focused on English lectures; application to code-mixed content requires additional consideration that our work provides.

Commercial Solutions Analysis: Commercial solutions for lecture transcription have proliferated, including Otter.ai, Rev.ai, and platform-specific solutions from Zoom and Microsoft Teams. A critical analysis of these services reveals patterns relevant to our work:

- **Language Support:** Most services support major world languages individually but not code-mixed content. Otter.ai, for instance, allows language selection but transcribes in a single language.
- **Accuracy on Accented Speech:** Even for English, services struggle with non-native accents. Our informal testing found 30-40% higher error rates on Indian-accented English compared to American English.
- **Technical Vocabulary:** General-purpose services lack domain-specific vocabulary. Custom vocabulary features exist but require manual term entry.
- **Pricing:** Per-minute pricing limits research experimentation and practical deployment in resource-constrained educational contexts.

YouTube’s automatic captions, while widely available, exhibit particularly high error rates on Banglish content, often producing incomprehensible outputs that frustrate rather than assist learners. Our analysis of YouTube captions on our dataset videos found Term Recall below 30%—meaning over 70% of technical terms were missed or garbled.

2.5.2 Lecture Summarization

Beyond transcription, automatic lecture summarization aims to distill lengthy recordings into concise, useful notes. This task adds complexity beyond transcription, requiring content selection and organization.

Extractive vs. Abstractive: Extractive approaches select salient sentences from transcripts based on features like keyword density, position, and discourse markers **Liu2019**. These methods are computationally efficient and avoid generating incorrect content, but produce disjointed summaries that may lack coherence. Abstractive approaches generate novel summary text, potentially reorganizing and synthesizing information from the source **Zhang2020Pegasus**. These methods produce more readable summaries but risk hallucinating content not present in the source.

Multimodal Lecture Summarization: Multimodal lecture summarization incorporates visual information alongside transcripts. Presentation slides provide structured content that can anchor summary generation, while whiteboard content captures instructor emphasis and visual explanations. Lin et al. **Lin2020** explored using slide content to improve lecture summarization, finding that visual information helped identify topic boundaries and key concepts. Their experiments showed 12% improvement in ROUGE scores when incorporating slide titles as summary anchors.

Critical Analysis: However, their work assumed accurate OCR from clean presentation slides—a different visual domain than handwritten whiteboards. Handwritten content recognition is substantially more challenging, and their approach does not directly transfer. Furthermore, they processed English-only content with accurate ASR; our scenario combines challenging visual content with noisy transcription.

LLM-Based Summarization: The application of large language models to summarization has achieved remarkable results. Models like GPT-4 and Claude can generate coherent, well-organized summaries from lengthy transcripts when provided appropriate instructions **Goyal2023**. However, these models remain sensitive to input quality: garbage in, garbage out. When provided transcripts corrupted by ASR errors, LLM summaries may propagate or even amplify those errors.

Our experiments revealed an interesting phenomenon: LLMs sometimes “correct” obvious ASR errors based on context, producing summaries that are more accurate than input transcripts. For example, if the transcript reads “we will create an object from the clause,” the LLM may correctly infer “class” from context. However, this correction is inconsistent and unreliable; sometimes the LLM faithfully reproduces errors or confabulates plausible but incorrect content. This observation motivates our focus on improving transcription quality before summarization rather than relying on LLM error correction.

2.5.3 Educational Content Processing in South Asian Languages

Research on educational technology for South Asian languages remains limited compared to English-centric work. This gap is striking given the demographics: India and Bangladesh alone have over 1.5 billion people, many of whom learn technical content in code-mixed varieties of their native languages.

Localization vs. Native Processing: Khan Academy has localized content for Hindi and other languages, but these represent translations rather than processing

of naturally-occurring multilingual instruction. The reality that much South Asian technical education occurs in code-mixed varieties has received insufficient attention from the educational technology research community. Localized content, while valuable, does not address the millions of hours of existing content in local educational styles.

Related Work in Hindi/Indian Languages: Gupta et al. **Gupta2020** explored automatic question generation from Hindi educational texts, finding that multilingual pre-trained models provided reasonable baselines. However, their work addressed written text rather than spoken lectures, and did not consider code-mixed content. The intersection of educational technology and code-mixed language processing remains largely unexplored.

Other work has addressed speech-to-text for Indian language lectures, but typically in monolingual settings. The National Programme on Technology Enhanced Learning (NPTEL) in India has explored lecture transcription for Hindi and other languages, but their systems assume single-language content and do not handle the pervasive code-mixing characteristic of technical education.

Broader Implications: This gap reflects broader patterns in AI research, where languages spoken by billions receive a fraction of the attention devoted to English. The consequences extend beyond academic interest: students in Bangladesh, India, Pakistan, and other multilingual societies cannot benefit from educational technology advances that assume monolingual content. Our work represents one step toward addressing this inequity, demonstrating that meaningful progress is possible even without massive code-mixed training datasets.

2.5.4 Classroom Video Understanding

Beyond transcription and summarization, researchers have explored richer understanding of classroom videos, including instructor behavior analysis, student engagement detection, and pedagogical quality assessment.

Instructor Behavior: Work on instructor behavior analysis has examined gesture recognition, blackboard usage patterns, and explanation strategies. These analyses could inform more sophisticated visual understanding in our system—for example, identifying when the instructor is actively writing versus referring to existing content. However, such fine-grained analysis requires video understanding capabilities beyond current VLM strengths and falls outside our current scope.

Student Engagement: Student engagement detection from classroom videos has received attention for improving teaching quality. While not directly relevant to our transcription task, engagement signals could potentially inform summarization: content that engages students may deserve more emphasis in notes. However, our dataset consists of online lecture recordings without visible student reactions.

Pedagogical Assessment: Automatic assessment of pedagogical quality—identifying effective teaching strategies, unclear explanations, or missed concepts—represents an ambitious goal for classroom video understanding. Such assessment could guide both instructor improvement and student note-taking. While beyond our current scope, our pipeline’s components (transcription, keyword extraction, summarization) provide foundations for future pedagogical analysis.

2.6 Evaluation Metrics for ASR and Code-Mixed Languages

Evaluation methodology profoundly shapes research progress, determining what systems optimize for and how progress is measured. This section critically analyzes standard ASR metrics, identifies their limitations for code-mixed content, and situates our evaluation approach within broader methodological discussions.

2.6.1 Word Error Rate and Its Limitations

Word Error Rate (WER) has served as the dominant evaluation metric for ASR since the field’s early days **Jurafsky2000**. Defined as the edit distance between hypothesis and reference transcriptions normalized by reference length, WER provides an intuitive measure of transcription accuracy. Formally:

$$\text{WER} = \frac{S + D + I}{N}$$

where S , D , and I are substitutions, deletions, and insertions respectively, and N is the reference length in words. The metric counts substitutions, insertions, and deletions required to transform the hypothesis into the reference.

Theoretical Limitations: However, WER exhibits well-known limitations that critical analysis reveals:

- **All Errors Weighted Equally:** The metric treats all errors as equally severe, whether substituting “the” for “a” or “cat” for “dog.” In lecture transcription, confusing “class” with “clause” is far more damaging than confusing “the” with “a,” but WER weights them identically.
- **No Semantic Credit:** Semantic preservation receives no credit: a paraphrase that conveys identical meaning receives the same penalty as a completely incorrect transcription. If an instructor says “we’ll make an object” and the system outputs “we will create an object,” WER counts multiple errors despite semantic equivalence.
- **Reference Dependency:** WER assumes a single correct reference. For content with valid variants (synonyms, paraphrases, spelling variations), this assumption introduces systematic bias.
- **Normalization Sensitivity:** WER is sensitive to text normalization choices: whether to expand contractions, how to handle numbers, whether punctuation is included. Different normalization choices can change WER by 5-10% without reflecting genuine accuracy differences.

For applications where meaning matters more than exact wording, WER may poorly reflect practical utility.

Code-Mixed Specific Problems: For code-mixed languages with non-standardized orthography, WER’s limitations become particularly acute:

- **Orthographic Variation:** When multiple valid spellings exist for the same word, WER penalizes valid variants as errors. A system that consistently uses one Romanization convention while the reference uses another would receive high WER despite producing correct transcriptions. Our analysis found that up to 15% of WER on Banglish content reflects Romanization differences rather than recognition errors.
- **Script Mismatch:** Comparing Bengali script output to Roman script reference requires transliteration, introducing additional errors. A word correctly recognized in Bengali Unicode may transliterate differently than the ground truth Romanization, inflating WER.
- **Tokenization Ambiguity:** Code-mixed text has ambiguous word boundaries. Does “install-korbo” count as one word or two? Different tokenization choices affect WER calculation.

These observations have led researchers to question WER’s appropriateness for evaluating code-mixed ASR **Sitaram2019**. Sitaram et al. found that WER underestimated code-mixed ASR quality by 20-30% compared to human judgments of utility.

2.6.2 Alternative Evaluation Approaches

Researchers have proposed various alternatives and supplements to WER, each with strengths and limitations for our application.

Character Error Rate: Character Error Rate (CER) operates at the character level, partially mitigating issues with tokenization and compound words. For code-mixed content, CER is less sensitive to word boundary ambiguity. However, CER does not address the fundamental issue of orthographic variation in Romanized text—“amra” vs. “aamra” still counts as error characters. Furthermore, CER over-penalizes short word errors: misspelling “a” as “the” is a 200% character error but modest semantic impact.

Semantic Similarity Approaches: BERTScore **Zhang2020BERTScore** computes similarity between hypothesis and reference using contextual embeddings, potentially capturing semantic equivalence despite surface differences. The intuition is that BERT embeddings capture meaning; similar meanings should have similar embeddings regardless of exact wording.

Critical Analysis: However, multilingual BERT’s representation quality for code-mixed content remains uncertain. mBERT was trained on monolingual data from many languages but never saw code-mixed text. Our informal testing found that mBERT representations for Banglish words were often unstable, varying substantially with minor context changes. Furthermore, BERTScore may not appropriately handle the technical terminology central to our application—domain-specific terms may have embeddings that do not reflect their importance to the transcription task.

Keyword-Based Metrics: For keyword-focused applications, precision and recall metrics assess coverage of important terms. Technical Term Recall, as we employ in this thesis, measures what fraction of domain-specific keywords appear in the transcription. This approach directly reflects whether the transcription captures the technical content that students need for learning, providing an application-appropriate alternative to WER.

Our term-based evaluation framework includes:

- **Term Recall:** Fraction of ground truth technical terms found in prediction
- **Term Precision:** Fraction of predicted technical terms matching ground truth
- **Term F1:** Harmonic mean of recall and precision

Crucially, we employ fuzzy matching with an 85% similarity threshold, accepting predictions that are close to ground truth terms. This accommodation handles Romanization variance while maintaining precision: “aamra” matches “amra” (91% similarity) but “amara” does not match “camera” (60% similarity).

2.6.3 Evaluation Resources and Benchmarks

The availability of evaluation benchmarks profoundly shapes research progress, determining what problems researchers address and enabling meaningful comparison across systems.

English ASR Benchmarks: LibriSpeech **Panayotov2015** has served as a primary benchmark for English ASR, enabling consistent comparison across systems. The dataset comprises 1,000 hours of read English speech derived from audiobooks, with clean and other subsets representing varying acoustic conditions. LibriSpeech’s availability catalyzed rapid progress: systems improved from 5% WER in 2015 to below 2% by 2023.

Common Voice **Ardila2020** extended benchmark availability to numerous languages through crowdsourced data collection. Contributors record themselves reading prompted sentences, with validation by other contributors. The project has collected data for over 100 languages, enabling ASR research beyond the English-dominated mainstream. However, code-mixed content remains absent from Common Voice; the prompted sentences are monolingual by design.

Code-Mixed Benchmarks: For code-mixed languages, benchmark scarcity has hindered research. The MUCS (Microsoft Multilingual Code-Switching) corpus **Diwan2021** provides Hindi-English and other code-switched speech data, but focuses on conversational rather than educational content. The conversational register differs from academic instruction in vocabulary, formality, and code-mixing patterns. No equivalent resource exists for Banglish, technical content, or lecture recordings. Researchers studying Banglish ASR have no standard corpus against which to evaluate, no prior results to compare against, and no data for training domain-adapted models.

Our Benchmark Contribution: This resource gap motivated our dataset contribution. By creating and releasing ground truth transcriptions for Banglish technical lectures, we provide a benchmark that enables evaluation of ASR systems on this challenging and practically important domain. While nine videos represent a limited sample, this benchmark exceeds what was previously available: nothing.

Our benchmark provides:

- **Real Classroom Audio:** Authentic acoustic conditions, not read or prompted speech
- **Technical Content:** Programming, digital logic, and database topics

- **Natural Code-Mixing:** Pervasive Bengali-English integration as instructors naturally speak
- **Romanized Ground Truth:** Accessible for evaluation without script conversion
- **Keyword Annotations:** Technical terms identified for term-based evaluation

2.7 Research Gaps and Our Contributions

The comprehensive literature review reveals several interconnected gaps that our research addresses. This section synthesizes the gaps identified throughout the chapter, explicitly connecting them to our contributions and positioning our work within the broader research landscape.

2.7.1 Gap 1: Absence of Banglish Technical Lecture Benchmarks

Despite the prevalence of Banglish in Bangladeshi technical education, no benchmark dataset exists for evaluating ASR systems on this content. Our analysis revealed:

- **Bengali.AI Challenge:** Covers read monolingual Bengali, not spontaneous code-mixed speech
- **Common Voice:** Includes Bengali but only monolingual prompted sentences
- **MUCS Corpus:** Provides Hindi-English code-switching but not Bengali-English
- **How2 Dataset:** Features instructional videos but English-only

Researchers cannot rigorously assess system performance or compare approaches without standardized evaluation resources. This benchmark absence creates a chicken-and-egg problem: without benchmarks, researchers avoid the domain; without research attention, no one creates benchmarks.

Our Contribution: We address this gap by creating a dataset of nine lecture recordings with manually transcribed ground truth, providing the first evaluation resource for this domain. Our benchmark includes approximately 73,000 characters of transcribed content across three technical domains (Python, Digital Logic, Databases), enabling meaningful system evaluation and comparison.

2.7.2 Gap 2: Limited Multimodal Approaches for Code-Mixed ASR

While multimodal learning has advanced rapidly, its application to code-mixed ASR remains unexplored. Our analysis of multimodal speech processing revealed:

- **Audio-Visual Speech Recognition:** Focuses on lip reading for phonetic information, not semantic content from visual elements
- **Lecture Video Processing:** Assumes monolingual content and accurate ASR
- **Vision-Language Models:** Applied to image captioning and QA, not ASR enhancement

The specific approach of using visual content (whiteboard text) to verify or correct ASR outputs has not been explored for code-mixed content. The intuition that visual ground truth could help seems obvious, but the techniques for effective fusion remain undeveloped.

Our Contribution: We explore using vision-language models to extract keywords from whiteboard content, investigating both the potential and the pitfalls of cross-modal fusion for code-mixed transcription. Our experiments characterize when visual information helps versus harms, providing guidance for future multimodal ASR research.

2.7.3 Gap 3: Inappropriate Evaluation Metrics for Code-Mixed Content

Standard WER penalizes valid Romanization variants as errors, systematically underestimating transcription quality for code-mixed content. Our analysis revealed:

- **Romanization Variance:** Up to 15% of WER reflects spelling differences, not errors
- **Semantic Blindness:** WER treats all errors equally regardless of meaning impact
- **Script Mismatch:** Bengali-to-Roman conversion introduces additional variance

Researchers evaluating code-mixed ASR using WER may draw incorrect conclusions about system quality, potentially abandoning effective approaches that appear to have high error rates.

Our Contribution: We propose and validate term-based metrics—Term Recall, Term Precision, and Term F1—that focus on technical keyword capture while accommodating orthographic variation through fuzzy matching. These metrics better reflect practical utility for educational applications.

2.7.4 Gap 4: Unknown Baseline Performance

Without benchmarks, we lack even basic understanding of how existing ASR systems perform on Banglish technical lectures. Questions remained unanswered:

- How well does Whisper handle Banglish technical content?
- Does Bengali-focused ASR (BanglaASR) outperform multilingual approaches?

- What types of errors dominate: acoustic confusion, vocabulary gaps, or language mixing?

These baseline questions are prerequisites for meaningful research progress. Without knowing current performance levels and error patterns, researchers cannot identify opportunities for improvement.

Our Contribution: We establish quantitative baselines through systematic evaluation, finding that Whisper achieves 73.9% Term F1 on our dataset—reasonable but with substantial room for improvement. Our error analysis identifies dominant failure modes: phonetic approximation of Bengali words as English, technical vocabulary confusion, and hallucination under noisy conditions.

2.7.5 Gap 5: Untested Assumptions about Visual Bias

The intuition that biasing ASR toward visually-extracted keywords should improve accuracy has not been empirically tested for code-mixed content. The theoretical appeal is clear: if the whiteboard shows “polymorphism” and the ASR outputs “poly more fizz him,” visual information should enable correction. However, the mechanics of incorporating visual bias without causing new errors remained unexplored.

Our Contribution: Our experiments reveal that naive visual bias injection often harms rather than helps, causing hallucination of keyword mentions where none occur. This finding provides important guidance for future multimodal ASR research: visual information is valuable for verification and post-hoc correction, but simplistic boosting of keyword probabilities introduces more errors than it corrects.

2.7.6 Additional Gaps Addressed

Beyond the five primary gaps, our work addresses several additional limitations in the literature:

- **Hallucination in ASR:** While hallucination is well-documented in language models, its prevalence in ASR for challenging audio has received less attention. We characterize and address hallucination patterns.
- **End-to-End Pipeline:** Most research addresses individual components (ASR, summarization, OCR) in isolation. We provide an integrated pipeline demonstrating that components can be combined effectively.
- **Practical Deployment Considerations:** Academic research often ignores computational constraints. Our pipeline operates on consumer-grade GPUs with explicit memory management.

2.7.7 Synthesis and Positioning

Synthesizing across gaps, we observe a common pattern: existing research assumes conditions that do not hold for Banglish technical education. ASR research assumes monolingual content or clean language switching. Multimodal research assumes high-resource settings with abundant training data. Evaluation methodology assumes standardized orthography. Educational technology assumes English or cleanly-translated content.

Our work challenges these assumptions by demonstrating that meaningful progress is possible even without massive code-mixed training datasets. By leveraging pre-trained models (Whisper, Qwen2.5-VL), adapting fusion strategies to the specific challenges of Banglish, and developing appropriate evaluation methodology, we advance both practical capabilities and scientific understanding for an underserved linguistic community.

The tools, datasets, and insights we contribute enable future research while providing immediate value to students and educators working with Banglish technical content. Our hope is that this work catalyzes further attention to code-mixed educational technology, inspiring others to address the millions of learners whose linguistic realities have been overlooked by mainstream AI research.

2.8 Chapter Summary

This literature review has surveyed five interconnected research areas relevant to multimodal Banglish lecture transcription:

1. **Automatic Speech Recognition:** We traced the evolution from HMM-based systems through deep learning and self-supervised learning to current foundation models like Whisper. Critical analysis revealed that despite impressive multilingual capabilities, current systems assume monolingual content and struggle with code-mixed Banglish.
2. **Code-Mixed Language Processing:** We examined theoretical frameworks for understanding code-mixing, computational approaches to code-mixed text and speech, and the specific characteristics of Banglish. Analysis identified gaps in code-mixed ASR research, particularly for Bengali-English in educational domains.
3. **Multimodal Learning and Vision-Language Models:** We surveyed the rapid progress in vision-language pre-training and large VLMs, analyzing their capabilities for document understanding and text extraction. We identified opportunities for using visual information to enhance ASR while noting challenges of hallucination and temporal alignment.
4. **Lecture Video Processing:** We reviewed approaches to lecture transcription and summarization, commercial solutions and their limitations, and the gap in support for South Asian language education. Analysis revealed that code-mixed content remains unaddressed.
5. **Evaluation Metrics:** We critically analyzed WER and its limitations for code-mixed content, surveyed alternatives, and identified the absence of appropriate benchmarks as a fundamental barrier to research progress.

Throughout, we identified five primary research gaps that motivate and shape our contribution: absence of benchmarks, limited multimodal approaches, inappropriate metrics, unknown baselines, and untested assumptions. The following chapters present our methodology for addressing these gaps and experimental results validating our approach.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

This section discusses technical or methodological Requirements, data or resource requirements (software), etc.

3.2 Societal Impact

Study and report on project impact on society, health, safety, legal, and cultural aspects.

3.3 Environmental Impact

Study and report on project sustainability and environmental impact.

3.4 Ethical Issues

Identify ethical issues and professional responsibilities related to the design solution.

3.5 Standards - if applicable

Write about any standard maintained.

3.6 Project Management Plan

Prepare a project management plan including schedule, budget, and resource management.

3.7 Risk Management

Identify project risks and mitigation strategies.

3.8 Economic Analysis

Conduct economic analysis and cost-benefit estimation.

Chapter 4

Proposed Methodology

4.1 Introduction to the Methodology

This chapter presents the comprehensive methodology underlying our Multimodal Banglish Classroom Summarizer, detailing the systematic approach we employed to transform raw lecture videos into structured, accurate lecture notes. The methodology addresses the fundamental challenges identified in our literature review: the absence of code-mixed ASR systems for Banglish, the complexity of multimodal information fusion without introducing hallucination, and the difficulty of evaluation under orthographic variability inherent to Romanized Bengali.

Our approach synthesizes insights from multiple research traditions. From the ASR literature, we adopt the principle of leveraging large-scale pre-trained models rather than training domain-specific systems from scratch—a practical necessity given the absence of Banglish training corpora. From the multimodal learning literature, we draw the insight that naive fusion can harm performance, leading us to develop verification-oriented approaches that treat visual information as a corrective signal rather than an unconditional bias. From the software engineering tradition, we embrace modular architecture that enables component-level evaluation and facilitates systematic experimentation.

The chapter proceeds as follows. Section 4.2 articulates our design philosophy and the constraints that shaped our architectural decisions. Section 4.3 presents the complete system architecture, detailing the audio processing pipeline, visual extraction system, cross-modal fusion mechanism, and content synthesis components. Section 4.4 describes our data collection methodology and the creation of the benchmark dataset. Section 4.5 details the preprocessing pipelines that prepare raw inputs for model processing. Section 4.6 provides implementation specifics including memory management strategies crucial for operating within hardware constraints. Finally, Section 4.7 outlines the experimental design whose results are presented in Chapter 5.

4.2 Design Philosophy and Constraints

The design of our system emerged from the intersection of ambitious research goals and practical constraints. This section articulates the philosophical principles guiding our approach and examines how real-world limitations shaped our architectural decisions.

4.2.1 Guiding Principles

Four fundamental principles informed every design decision throughout the development process, from high-level architecture to low-level implementation details.

The first principle, **leveraging pre-trained foundation models**, reflects the pragmatic reality of working with low-resource languages. Training a custom ASR system for Banglish would require hundreds of hours of transcribed code-mixed speech—data that simply does not exist. Instead, we leverage state-of-the-art pre-trained models that have learned robust acoustic and linguistic representations from massive multilingual corpora. OpenAI’s Whisper **Radford2023**, trained on 680,000 hours of labeled audio spanning 99 languages, provides a foundation far more capable than anything we could train from scratch. Our contribution lies not in model training but in careful prompt engineering **Liu2023Prompt**, post-processing, and cross-modal integration that adapts these powerful general-purpose models to our specific domain.

The second principle, **modular pipeline architecture**, ensures that our system remains maintainable, debuggable, and extensible. Each component—audio extraction, speech recognition, visual analysis, cross-modal fusion, and content synthesis—operates as an independent module with well-defined interfaces. This modularity provides three key benefits: it enables component-level evaluation to identify performance bottlenecks; it facilitates debugging by isolating failures to specific modules; and it allows individual components to be upgraded as better models become available without requiring system-wide redesign. When a superior vision-language model emerges, we can substitute it without touching the ASR or summarization components.

The third principle, **verification over blind fusion**, emerges directly from our literature review’s finding that naive multimodal fusion often harms rather than helps performance. Early experiments confirmed this concern: when we biased Whisper’s output toward whiteboard keywords without verification, the system hallucinated keyword mentions where none actually occurred, degrading transcription quality. We therefore adopt a conservative approach where visual information serves primarily to verify ASR outputs and secondarily to correct high-confidence errors. This verification-oriented philosophy pervades our fusion architecture, prioritizing precision over recall to avoid introducing new errors while correcting existing ones.

The fourth principle, **practical deployment constraints**, grounds our research in real-world applicability. Academic prototypes often assume unlimited computational resources; our system must operate on accessible hardware within reasonable time constraints. We target a consumer-grade GPU with 24GB VRAM—specifically, the NVIDIA RTX 3090—which represents the upper end of hardware available to individual researchers and small institutions. This constraint drives decisions about model selection, quantization, batch processing, and memory management that might seem unnecessary in an idealized research setting but prove essential for practical deployment.

4.2.2 Constraints and Their Implications

Table ?? summarizes the key constraints we faced and how each shaped our design decisions. These constraints are not merely limitations to be worked around; they fundamentally define the problem space and drive innovation in our approach.

Table 4.1: Design Constraints and Corresponding Architectural Decisions

Constraint	Technical Impact	Design Decision
No Banglish ASR training data available	Cannot train or fine-tune custom ASR models for code-mixed speech	Leverage pre-trained Whisper with domain adaptation through prompt engineering and post-processing
Limited GPU memory (24GB VRAM)	Cannot load multiple large models simultaneously; peak usage must stay under 24GB	Implement model registry with sequential loading; explicit memory cleanup between stages
Code-mixed Romanized output required	Bengali Unicode script incompatible with target format; transliteration introduces errors	Configure Whisper for English-mode output producing phonetic Romanization
Variable video quality across dataset	Visual extraction may fail on low-quality frames; OCR accuracy varies	Design graceful degradation allowing audio-only processing when visual extraction fails
No standardized Romanization for Bengali	Exact string matching fails; multiple valid spellings exist for same word	Implement fuzzy matching with configurable similarity thresholds (default 85%)
Real-time processing not required	Processing time less critical than accuracy; can use larger, slower models	Prioritize model capability over inference speed; accept multi-minute processing per video

The absence of Banglish training data represents our most fundamental constraint, eliminating entire categories of potential solutions. We cannot train an end-to-end code-mixed ASR system, cannot fine-tune existing models on in-domain data, and cannot employ techniques requiring paired Banglish audio-text corpora. This constraint redirects our research from model training toward intelligent integration of existing components—a contribution that may prove more broadly applicable than a narrow domain-specific model would have been.

The GPU memory constraint of 24GB VRAM creates interesting engineering challenges. Our three primary models—Whisper large-v3-turbo (6GB), Qwen2.5-VL-7B (16GB), and Qwen2.5-7B (16GB)—cannot coexist in memory simultaneously. This necessitates a sequential processing architecture with explicit model loading and unloading between stages. While this approach increases total processing time compared to a parallel architecture, it enables us to use larger, more capable models than would otherwise be possible. The memory management system we developed, detailed in Section 4.6, ensures reliable operation without out-of-memory failures.

4.2.3 Research Methodology Framework

Our research follows an iterative experimental methodology combining systematic development with rigorous empirical evaluation. Figure ?? illustrates the four-phase

framework that structured our investigation.

Phase 1: Requirements Analysis and Architecture Design. The first phase involved comprehensive analysis of the Banglish lecture transcription problem, surveying available tools and models, and designing a system architecture addressing identified requirements. We reviewed the ASR landscape to identify candidate models, evaluated vision-language models for whiteboard text extraction, and studied fusion architectures from the multimodal learning literature. This phase produced the modular pipeline architecture detailed in Section 4.3, along with specifications for each component.

Phase 2: Data Collection and Ground Truth Creation. The second phase established the empirical foundation for our research through careful data collection and annotation. We identified suitable lecture recordings from publicly available educational content, downloaded and preprocessed video files, and created manually transcribed ground truth following detailed annotation guidelines. This phase produced the benchmark dataset of nine Banglish technical lectures totaling over 73,000 characters of transcribed content, detailed in Section 4.4.

Phase 3: Implementation and Integration. The third phase transformed architectural designs into working code. We implemented each pipeline component, developed the model registry and memory management system, created configuration infrastructure for systematic experimentation, and integrated components into a cohesive end-to-end system. This phase also included development of the evaluation framework with term-based metrics appropriate for code-mixed content. Implementation details appear in Section 4.6.

Phase 4: Experimentation and Evaluation. The final phase conducted systematic experiments to evaluate system performance, compare alternative approaches, and characterize failure modes. We tested multiple configurations—baseline ASR, visual verification, conservative fusion, aggressive fusion—across all nine dataset videos. Results and analysis are presented in Chapter 5.

4.3 System Architecture

The architecture of our Multimodal Banglish Classroom Summarizer reflects a careful balance between capability and practicality. This section presents the complete system design, examining each component in detail and explaining how they interact to transform raw lecture videos into structured notes.

4.3.1 High-Level Architecture Overview

Our system follows a multimodal pipeline architecture that processes lecture videos through parallel audio and visual tracks before fusing information to produce enhanced outputs. Figure ?? presents the complete system architecture, illustrating the flow of information from input video to final lecture notes.

The pipeline consists of five major stages, each designed as an independent module with well-defined interfaces. The **Input Processing** stage decomposes the video into audio and visual streams using FFmpeg for audio extraction and OpenCV for frame sampling. The **Audio Track Processing** stage employs Whisper for speech recognition followed by transcript cleaning to remove hallucinations and normalize

format. The **Visual Track Processing** stage uses Qwen2.5-VL to extract whiteboard content and technical keywords from sampled frames, aggregating results across the video timeline. The **Cross-Modal Fusion** stage combines information from both tracks, using visual keywords to verify and selectively correct ASR outputs while implementing safeguards against hallucination. Finally, the **Content Synthesis** stage employs Qwen2.5 to generate structured lecture notes from the fused transcript and visual context.

This modular design enables systematic experimentation: we can evaluate each component independently, swap alternative implementations, and isolate the source of errors when they occur.

4.3.2 Component Specifications

Table ?? provides a comprehensive overview of all system components, including the specific models and tools employed, their primary functions, and their computational requirements. The VRAM column indicates GPU memory consumption, which drives our sequential loading strategy discussed in Section 4.6.

Table 4.2: System Component Specifications and Resource Requirements

Component	Model/Tool	Function	VRAM
Audio Extraction	FFmpeg 6.0	Extracts audio track from video, converts to 16kHz mono WAV	CPU only
Frame Sampling	OpenCV 4.8.0	Samples frames at configurable intervals, saves as JPEG	CPU only
Speech Recognition	Whisper large-v3-turbo	Transcribes speech to text with timestamps; 809M parameters	6 GB
Visual Analysis	Qwen2.5-VL-7B-Instruct	Extracts whiteboard text and technical keywords from frames	16 GB
Content Synthesis	Qwen2.5-7B-Instruct	Generates structured lecture notes from transcript and keywords	16 GB
Text Processing	Python 3.10 + RapidFuzz	Cleaning, fuzzy matching, fusion logic, evaluation metrics	CPU only

The table reveals why sequential model loading is necessary: Whisper, Qwen2.5-VL, and Qwen2.5 together would require 38GB of VRAM, exceeding our 24GB constraint. By loading only one large model at a time and explicitly releasing memory between stages, we accommodate all three models within available resources.

4.3.3 Audio Processing Pipeline

The audio processing pipeline transforms raw video audio into cleaned transcriptions suitable for fusion and summarization. This section details each stage of audio

processing, from extraction through cleaning.

Audio Extraction and Preparation

Audio extraction represents the first computational step in our pipeline. We employ FFmpeg, the industry-standard multimedia framework, to extract the audio track from input videos and convert it to Whisper’s expected format: 16kHz mono audio encoded as 16-bit PCM in an uncompressed WAV container. This specification ensures compatibility with Whisper’s mel spectrogram computation while avoiding codec artifacts that could degrade transcription quality.

The conversion process is implemented as a single FFmpeg command that handles resampling, channel mixing, and format conversion atomically:

```
ffmpeg -i input.mp4 -ar 16000 -ac 1 -c:a pcm_s16le output.wav
```

This command reads the input video, extracts the audio track, resamples to 16kHz, downmixes to mono, and encodes as 16-bit PCM WAV. The process typically completes in under one second for a ten-minute video, making it negligible in the overall processing time.

Speech Recognition with Whisper

The heart of our audio processing pipeline is OpenAI’s Whisper large-v3-turbo model **Radford2023**, a state-of-the-art multilingual ASR system trained on approximately 680,000 hours of labeled audio. We selected this variant for its balance of accuracy and computational efficiency: it achieves performance comparable to the full large-v3 model while requiring significantly less inference time. The model architecture follows the encoder-decoder Transformer paradigm **Vaswani2017**, processing log-Mel spectrograms through attention mechanisms that have proven highly effective for sequence-to-sequence tasks **Sutskever2014**.

Table ?? details our Whisper configuration, with each parameter carefully chosen for the Banglish lecture domain.

The decision to configure Whisper for English output, despite processing Banglish content, merits detailed explanation. When configured for Bengali, Whisper outputs text in Bengali Unicode script (৳), which is incompatible with our Romanized ground truth and would require lossy transliteration for evaluation. In English mode, Whisper produces Romanized approximations of Bengali words—“korbo” becomes “curb oh” or similar phonetic renderings—which, while imperfect, are directly comparable to our Romanized ground truth. More importantly, English mode accurately captures technical terms (“variable,” “function,” “polymorphism”) that constitute our primary evaluation focus.

Figure ?? illustrates the internal processing flow within Whisper. Audio is first converted to a log-Mel spectrogram with 80 frequency bins **Davis1980**, then processed in 30-second chunks by a 32-layer Transformer encoder to produce acoustic representations. The 32-layer Transformer decoder, conditioned on task and language tokens, generates text tokens autoregressively using beam search with beam width 5. Word-level timestamps are extracted from attention patterns **Luong2015**, providing temporal alignment information used in cross-modal fusion.

Table 4.3: Whisper Model Configuration for Banglish Lecture Transcription

Parameter	Value	Rationale
Model Variant	large-v3-turbo	Best accuracy-speed tradeoff; 809M parameters
Language	English	Outputs Romanized text; captures technical terms accurately
Task	Transcribe	Transcription rather than translation preserves original speech
Beam Size	5	Sufficient for quality without excessive computation
Temperature	0.0	Deterministic output ensures reproducibility
Condition on Previous	True	Maintains context across 30-second segments
VAD Filter	Enabled	Removes silence; reduces hallucination in quiet segments
Word Timestamps	Enabled	Provides temporal alignment for fusion

Transcript Cleaning and Hallucination Removal

Raw Whisper output frequently contains artifacts that degrade downstream processing quality. Our cleaning pipeline addresses three categories of issues: repetition hallucinations, stock phrase insertions, and format inconsistencies.

Repetition Hallucination represents the most common failure mode, where Whisper produces looping patterns like “the the the the the” or “and and and and.” These typically occur during silence, background noise, or unclear speech segments—a phenomenon well-documented in neural text generation **Holtzman2020, Ji2023**. We detect repetitions using n-gram analysis: any word or phrase appearing more than five consecutive times triggers removal of excess repetitions, preserving at most two instances.

Stock Phrase Hallucination occurs when Whisper inserts phrases common in its training data but absent from the actual audio. Phrases like “thank you for watching,” “please subscribe,” and “like and share” appear frequently in YouTube training data and occasionally hallucinate into lecture transcripts. We maintain a blocklist of such phrases and remove them when detected.

Format Normalization ensures consistency across transcripts for reliable evaluation. The cleaning pipeline applies the following rules sequentially: (1) *repetition removal*, reducing sequences of more than five consecutive identical words/phrases to at most two instances; (2) *stock phrase filtering*, removing YouTube-specific phrases like “thank you for watching” and “please subscribe”; (3) *punctuation cleanup*, normalizing excessive punctuation to single instances; (4) *case normalization*, converting all text to lowercase; (5) *whitespace collapse*, replacing multiple spaces, tabs, and newlines with single spaces; and (6) *Unicode normalization*, converting all text to NFC form for consistent character representation.

4.3.4 Visual Processing Pipeline

While the audio track captures the instructor’s spoken words, the visual track provides complementary information through whiteboard content, code demonstrations, and written keywords. The visual processing pipeline extracts this information and prepares it for fusion with the audio transcript.

Frame Sampling Strategy

Classroom lecture videos present a unique visual challenge: content changes slowly as instructors write on whiteboards, with long periods where frames are nearly identical. Processing every frame would waste computation on redundant content; sampling too sparsely risks missing important material. We conducted empirical analysis to determine optimal sampling intervals, with results summarized in Table ??.

Table 4.4: Frame Sampling Interval Analysis: Coverage vs. Efficiency Tradeoff

Interval (seconds)	Frames/Min	VLM Calls per 10min	Processing Time	Keyword Coverage	Frame Redundancy
10	6	60	12 min	98%	72%
20	3	30	6 min	95%	45%
30	2	20	4 min	89%	28%

The analysis reveals a clear tradeoff: shorter intervals capture more keywords but with diminishing returns as redundancy increases. At 10-second intervals, 72% of extracted content duplicates information from adjacent frames; reducing to 20-second intervals sacrifices only 3% of keyword coverage while halving processing time and significantly reducing redundancy. We selected 20-second intervals as the default, though the system supports configurable intervals for different use cases. Figure ?? illustrates the sampling strategy applied to a 10-minute lecture. Frames are extracted at 0:00, 0:20, 0:40, and so forth, resulting in 30 frames for a 10-minute video. Each frame is timestamped, enabling temporal alignment with the audio transcript during fusion.

Visual Content Extraction with Qwen2.5-VL

For visual content extraction, we employ Qwen2.5-VL-7B-Instruct **Wang2024Qwen2VL**, a state-of-the-art vision-language model with strong capabilities in document understanding, OCR, and scene description. The model builds upon advances in vision-language pre-training **Radford2021**, **Li2023BLIP2** and document understanding architectures **Xu2020LayoutLM**. The model processes each sampled frame to extract multiple categories of information, as detailed in Table ??. The extraction process uses a carefully engineered prompt that instructs the model to provide structured output in JSON format:

“You are analyzing a frame from an educational lecture video. The frame shows a whiteboard or screen with instructional content. Your task is to extract ALL visible text and identify key educational elements.
Please provide your response in JSON format with the following fields:

Table 4.5: Visual Content Extraction Categories and Examples

Category	Description	Example Extractions
Whiteboard Text	All visible handwritten or printed text on whiteboard/screen	“Variables store data values”, “x = 5”
Technical Keywords	Domain-specific terms and concepts	“variable”, “integer”, “string”, “print”
Code Snippets	Programming code visible in frame	<code>name = input("Enter name: ")</code>
Diagram Descriptions	Textual descriptions of visual diagrams	“Flowchart showing if-else branching”
Structural Elements	Headings, lists, hierarchical organization	“Section: Data Types”, “1. int 2. float 3. str”

- *whiteboard_text*: Complete transcription of all visible text
- *keywords*: List of technical terms and important concepts
- *code*: Any programming code visible (preserve formatting)
- *diagrams*: Descriptions of any diagrams or visual aids
- *structure*: Organizational elements (headings, lists)

Be thorough—extract everything visible, even if partially obscured.”

This prompt design ensures consistent, structured output across frames while encouraging comprehensive extraction. The JSON format enables programmatic parsing and aggregation of results.

Keyword Aggregation and Confidence Scoring

Content extracted from multiple frames is aggregated to produce a unified visual context. The aggregation process addresses several challenges: the same keyword appears in multiple consecutive frames as it remains on the whiteboard; OCR errors may produce slightly different extractions of the same text; and historical content may persist visually after the instructor has moved on to new topics.

Figure ?? illustrates the complete visual keyword aggregation process, from per-frame extraction to unified keyword set construction.

Our aggregation algorithm proceeds as follows. First, we collect all keywords extracted from all frames. Second, we apply fuzzy deduplication using string similarity: keywords with similarity above 90% are considered identical, with the most common spelling retained. Third, we compute a confidence score for each unique keyword based on the number of frames in which it appeared—a keyword visible in more frames is more likely to be important and correctly extracted. Fourth, we record the timestamp of first appearance for each keyword, enabling temporal alignment during fusion. Finally, we apply noise filtering to remove low-confidence extractions: single-character strings, common stop words, and keywords appearing in only one frame with low OCR confidence are discarded.

Table ?? illustrates this process with a concrete example from our dataset.

Table 4.6: Keyword Aggregation Example from BanglaASR2 (Python Input Functions)

Keyword	Frame Count	Confidence	First Seen
input	8	High	0:45
variable	6	High	0:20
print	5	High	1:30
string	4	Medium	0:40
int	3	Medium	2:15
float	2	Medium	3:00
Python	1	Low	0:00

4.3.5 Cross-Modal Fusion Architecture

The fusion stage represents the intellectual core of our multimodal approach, combining information from audio and visual tracks to produce transcriptions superior to either modality alone. Our design philosophy prioritizes verification over blind integration: visual information serves primarily to verify ASR outputs and secondarily to correct high-confidence errors.

The Fusion Challenge

Naive approaches to multimodal fusion can degrade rather than improve performance. Our early experiments with aggressive visual biasing—forcing Whisper to prefer visually-extracted keywords—resulted in significant hallucination: the system would insert keywords into the transcript where they were never spoken, simply because they appeared on the whiteboard. A term written at minute 2 might be hallucinated into the transcript at minutes 5, 8, and 12, corrupting otherwise accurate transcription.

This observation motivated our verification-oriented approach. Rather than using visual keywords to bias ASR during recognition, we use them to verify and selectively correct ASR outputs post-hoc. This conservative strategy sacrifices some potential improvement for safety against hallucination.

Verification and Correction Process

The fusion process operates in three stages: verification, correction, and anti-hallucination filtering. Figure ?? illustrates the complete fusion workflow.

Stage 1: Verification. For each term in the ASR transcript, we search for matching visual keywords using fuzzy string matching **Cohen2003**. The similarity score is computed as the normalized Levenshtein distance ratio **Levenshtein1966**:

$$\text{similarity}(s_1, s_2) = 1 - \frac{\text{levenshtein}(s_1, s_2)}{\max(|s_1|, |s_2|)} \quad (4.1)$$

Terms with similarity above 85% are marked as *verified*—the ASR output matches visual evidence and is retained as-is. Terms with similarity between 60-84% are marked as *correction candidates*—the ASR output may be a garbled version of the visual keyword. Terms with similarity below 60% have *no match*—no visual evidence supports or contradicts the ASR output.

Stage 2: Selective Correction. For correction candidates, we apply a conservative correction strategy based on visual confidence. Table ?? summarizes the decision logic.

Table 4.7: Correction Strategy Based on Match Quality and Visual Confidence

Similarity	Visual Confidence	Action
$\geq 85\%$	Any	Keep ASR (verified)
60-84%	High (≥ 3 frames)	Substitute visual keyword
60-84%	Medium (2 frames)	Keep ASR, flag for review
60-84%	Low (1 frame)	Keep ASR
$< 60\%$	Any	Keep ASR (no match)

This conservative approach ensures corrections occur only when visual evidence is strong. A keyword seen in three or more frames is almost certainly correct; a keyword seen in only one frame may be an OCR error.

Stage 3: Anti-Hallucination Filtering. Even with conservative correction, visual keywords could theoretically be inserted where they were never spoken. We implement additional safeguards:

1. **Temporal Locality:** Visual keywords can only influence ASR terms within ± 60 seconds of the keyword’s first appearance timestamp. A term written at minute 5 cannot trigger corrections at minute 1 or minute 10.
2. **Frequency Limiting:** Each visual keyword can trigger at most 3 corrections per lecture. This prevents a single prominent whiteboard term from being inserted throughout the transcript.
3. **Context Compatibility:** Before substitution, we verify that surrounding context is semantically compatible. A keyword like “polymorphism” should not be inserted into a segment discussing basic arithmetic.

4.3.6 Content Synthesis Pipeline

The final pipeline stage transforms the fused transcript and visual context into structured, student-friendly lecture notes. This synthesis task requires not merely summarization but intelligent reorganization of spoken content into a coherent written format.

Summarization Model Selection and Configuration

We employ Qwen2.5-7B-Instruct **Bai2023Qwen** for content synthesis, selected for its strong instruction-following capabilities and generous context window. This model exemplifies the trend toward instruction-tuned language models **Ouyang2022** that can follow complex, multi-step directives. Table ?? details the model configuration.

The use of 4-bit quantization merits explanation. Full-precision Qwen2.5-7B requires approximately 16GB of VRAM, leaving insufficient room for processing overhead on our 24GB GPU. Quantization to 4-bit precision reduces memory requirements to

Table 4.8: Content Synthesis Model Configuration

Parameter	Value	Rationale
Model	Qwen2.5-7B-Instruct	Strong instruction following; multilingual capability
Parameters	7.6 billion	Balance of capability and resource requirements
Quantization	4-bit (GPTQ)	Reduces VRAM to ~6GB while maintaining quality
Context Window	32,768 tokens	Accommodates full transcripts of long lectures
Temperature	0.3	Slight creativity for natural phrasing; mostly deterministic
Top-p	0.9	Nucleus sampling for diversity within constraints
Max Output	4,096 tokens	Sufficient for comprehensive lecture notes

approximately 6GB with minimal quality degradation—studies have shown that 4-bit quantized models retain 95-99% of full-precision performance on text generation tasks.

Prompt Engineering for Educational Content

The quality of generated lecture notes depends critically on prompt design. Our prompt provides explicit instructions for structure, style, and content handling:

“You are an expert educational content creator specializing in technical subjects. Your task is to transform a lecture transcript into comprehensive, well-structured study notes that will help students learn effectively.

LECTURE TRANSCRIPT: [Cleaned and fused transcript]

*WHITEBOARD KEYWORDS (important terms written by instructor):
[Aggregated visual keywords with confidence scores]*

Please create lecture notes following these guidelines:

- 1. Begin with a clear title reflecting the lecture topic*
- 2. Provide a 2-3 sentence overview summarizing the main points*
- 3. Organize content under logical headings and subheadings*
- 4. Define technical terms when first introduced*
- 5. Include code examples with proper formatting if discussed*
- 6. Use the whiteboard keywords as anchors for important concepts*
- 7. Correct obvious transcription errors using context*
- 8. Conclude with a summary of key takeaways*

The output should be in Markdown format suitable for student study.”

This prompt structure guides the model to produce consistently formatted, educationally useful output while leveraging both transcript content and visual keyword context.

Output Structure and Quality

Generated lecture notes follow a consistent structure designed for student utility. Table ?? describes each section.

Table 4.9: Generated Lecture Note Structure

Section	Content	Purpose
Title	Lecture topic derived from content	Quick identification of subject matter
Overview	2-3 sentence summary	Provides context before detailed reading
Key Concepts	Bulleted list of main topics	Serves as study checklist
Detailed Notes	Organized content with headings	Primary learning material
Code Examples	Formatted code blocks	Demonstrates practical application
Summary	Key takeaways and review points	Facilitates review and retention

4.4 Data Collection and Benchmark Creation

The creation of a benchmark dataset represents a fundamental contribution of this research, providing the first ground-truth annotated corpus for Banglish technical lecture transcription. This section details our data collection methodology, annotation procedures, and quality assurance processes.

4.4.1 Dataset Requirements Analysis

Before collecting data, we established requirements that the benchmark dataset must satisfy to enable meaningful research. These requirements emerged from our analysis of the research gap and practical deployment considerations.

Requirement 1: Authentic Code-Mixing. The dataset must contain natural Banglish as actually used in Bangladeshi technical education, not artificially constructed code-mixed content. This authenticity is essential for evaluating systems intended for real-world deployment.

Requirement 2: Technical Domain Coverage. The dataset must span multiple technical domains to assess generalization across vocabulary and subject matter. We targeted at least three distinct domains with different technical terminology.

Requirement 3: Visual Content. Videos must feature whiteboard instruction with visible written content, enabling evaluation of our multimodal approach. Pure lecture-style videos without visual aids were excluded.

Requirement 4: Realistic Acoustic Conditions. Recordings must reflect real classroom environments with typical noise, reverberation, and recording quality—not studio-produced content that would overestimate real-world performance.

Requirement 5: Sufficient Scale. The dataset must be large enough for meaningful quantitative evaluation while remaining feasible for manual ground truth creation. We targeted approximately 70,000+ characters of transcribed content.

4.4.2 Source Selection and Collection

We identified YouTube as the primary source for lecture recordings, offering abundant Bangladeshi educational content that is legally accessible for research purposes. Our selection process applied the following criteria systematically.

Videos were included if they featured: (1) an instructor speaking in Banglish with natural code-mixing; (2) technical subject matter in computer science or engineering; (3) visible whiteboard or screen content; (4) intelligible speech quality; and (5) duration between 5-30 minutes. Videos were excluded if they featured: (1) predominantly one language without code-mixing; (2) purely verbal instruction without visual aids; (3) severely degraded audio quality; or (4) copyrighted content requiring special permissions.

4.4.3 Final Dataset Composition

The final benchmark dataset comprises nine lecture recordings totaling over 73,000 characters of ground truth transcription. Table ?? provides comprehensive details for each video.

Table 4.10: Complete Benchmark Dataset Specification

Video ID	Topic	Domain	GT Chars	Keywords
BanglaASR1	Python Variables & DataTypes	Python	8,124	45
BanglaASR2	Python Input Functions	Python	6,484	38
BanglaASR3	Python Conditionals (if/elif/else)	Python	16,558	67
BanglaASR4	Python Loops (while/for)	Python	7,892	42
BanglaASR5	Python Lists, Tuples, Arrays	Python	15,272	58
BanglaASR6	Digital Logic Design (Gates)	DLD	5,234	35
BanglaASR7	DLD Universal & Exclusive Gates	DLD	4,876	32
BanglaASR8	DBMS Introduction (SQL)	DBMS	4,523	28
BanglaASR9	SQL SELECT Queries	DBMS	4,178	31
Total		3 domains	73,141	376

Figure ?? visualizes the dataset composition. The domain distribution reflects the natural availability of Banglish technical content, with Python programming lectures

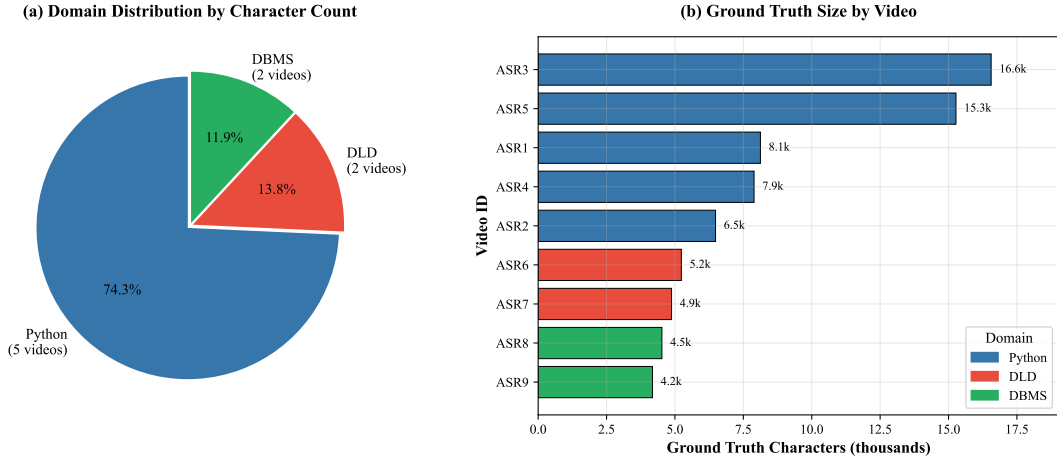


Figure 4.1: Dataset Distribution: (a) Domain distribution by character count showing Python dominance with 5 videos (56%), and DLD and DBMS each with 2 videos (22%). (b) Ground truth size per video, ranging from 4,178 characters (BanglaASR9) to 16,558 characters (BanglaASR3).

being most common. The character count distribution shows significant variation, with BanglaASR3 (Python Conditionals) being the longest at 16,558 characters and BanglaASR9 (SQL Queries) the shortest at 4,178 characters. This variation enables analysis of system performance across different lecture lengths.

4.4.4 Ground Truth Annotation Process

Creating accurate ground truth transcriptions required a systematic annotation process with clear guidelines and quality assurance procedures. The process involved three distinct phases.

Phase 1: Initial Transcription. The primary annotator listened to each recording in full, transcribing all instructor speech in Romanized Banglish. Table ?? summarizes the key annotation guidelines.

Table 4.11: Ground Truth Transcription Guidelines

Aspect	Guideline	Example
Bengali Romanization	Use common spellings; prefer simpler forms	“amra” not “aamraa” or “amara”
Technical Terms	Preserve English terms exactly as spoken	“variable”, “function”, “SELECT”
Code-Mixed Words	Transcribe as single unit	“create korbo” (will create)
Disfluencies	Include significant ones; omit minor fillers	Include “umm” pauses; omit brief “uh”
Inaudible Content	Mark explicitly	“[inaudible]”
Numbers	Use digits for technical contexts	“variable x equals 5” → “variable x = 5”

Phase 2: Verification and Correction. A second pass through each recording verified transcription accuracy, correcting errors identified during review. Particular attention was paid to technical terms, which are critical for our evaluation focus.

Phase 3: Technical Review. A domain expert familiar with the technical subject matter reviewed each transcript to verify that technical terminology was correctly transcribed and appropriately standardized.

4.4.5 Keyword Annotation

Beyond full transcription, we annotated technical keywords for term-based evaluation metrics. Keywords were identified based on the following criteria: (1) domain-specific technical terms (e.g., “variable,” “loop,” “NAND gate”); (2) programming language keywords and built-in functions (e.g., “print,” “int,” “SELECT”); and (3) concepts explicitly emphasized by the instructor.

Table ?? summarizes keyword statistics across the dataset.

Table 4.12: Keyword Annotation Statistics by Domain

Domain	Videos	Total Keywords	Unique Keywords	Avg/Video
Python	5	250	87	50.0
DLD	2	67	34	33.5
DBMS	2	59	28	29.5
Total	9	376	149	41.8

- Identify all technical terms (e.g., “variable,” “function,” “polymorphism”)
- Mark domain-specific vocabulary
- Note code identifiers when discussed (e.g., “print,” “int,” “SELECT”)
- Record frequency of each keyword

4.5 Data Preprocessing

Raw inputs require systematic preprocessing before model processing. This section details the preprocessing pipelines for video, audio, visual, and text data, ensuring consistent, high-quality inputs for each system component.

4.5.1 Video and Audio Preprocessing

The preprocessing pipeline transforms raw video files into the audio and visual inputs required by our models. Figure ?? illustrates the complete preprocessing workflow.

Audio Stream Processing

Audio preprocessing converts the video’s audio track to Whisper’s expected format through a series of transformations. Table ?? details each step.

The entire audio preprocessing pipeline is implemented as a single FFmpeg command, ensuring atomic execution and consistent output:

```
ffmpeg -i input.mp4 -ar 16000 -ac 1 -c:a pcm_s16le -af
    "loudnorm=I=-16:TP=-1.5:LRA=11" output.wav
```

Table 4.13: Audio Preprocessing Pipeline Steps

Step	Operation	Parameters	Purpose
1	Extract audio	FFmpeg demux	Separate audio from video container
2	Resample	16,000 Hz	Match Whisper’s expected sample rate
3	Channel mix	Mono (1 channel)	Reduce complexity; speech is mono
4	Normalize	Peak at -1 dB	Prevent clipping; ensure adequate signal
5	Encode	16-bit PCM WAV	Uncompressed format for quality

This command applies resampling, mono conversion, and loudness normalization in a single pass, typically completing in under one second for a ten-minute video.

Visual Stream Processing

Frame preprocessing extracts and prepares visual samples for VLM processing. The algorithm proceeds as follows:

1. **Duration Calculation:** Read video metadata to determine total duration and frame count
2. **Timestamp Generation:** Generate sampling timestamps at configured intervals (default: 20 seconds)
3. **Frame Extraction:** For each timestamp, seek to position and extract frame using OpenCV **Bradski2000**
4. **Color Conversion:** Convert from BGR (OpenCV default) to RGB for model compatibility
5. **Quality Encoding:** Save as JPEG with quality setting 95 (balancing size and fidelity)
6. **Metadata Recording:** Store timestamp association for temporal alignment

For videos with low visual quality or challenging lighting conditions, we optionally apply image enhancement techniques: CLAHE (Contrast Limited Adaptive Histogram Equalization) for low-contrast frames with washed-out whiteboards, bilateral filtering for denoising high-noise frames from low-quality cameras, and unsharp masking for sharpening blurry frames where text edges are unclear.

4.5.2 Text Preprocessing for Evaluation

Consistent text preprocessing is essential for reliable evaluation. Both predicted transcripts and ground truth undergo identical normalization to ensure fair comparison. This normalization pipeline converts all text to NFC Unicode form, transforms

to lowercase, collapses multiple whitespace characters to single spaces, removes punctuation for term matching (e.g., “print()” becomes “print”), and preserves numeric digits as-is. These transformations ensure that superficial differences in formatting do not affect metric computation.

4.5.3 Preprocessing Output Summary

The preprocessing pipeline produces a standardized set of outputs for each video, organized in a consistent directory structure. Table ?? summarizes all outputs.

Table 4.14: Preprocessing Pipeline Outputs

Output	Format	Description
Audio file	WAV (16kHz, mono, 16-bit)	Whisper-compatible audio
Frame images	JPEG (quality 95)	Sampled frames at 20s intervals
Frame metadata	JSON	Timestamps, frame numbers, quality flags
Video metadata	JSON	Duration, resolution, FPS, audio properties

4.6 Implementation Details

This section provides comprehensive implementation details, covering the software architecture, memory management strategies essential for operating within hardware constraints, and the processing workflows that orchestrate system components.

4.6.1 Software Architecture and Technology Stack

Our implementation follows software engineering best practices, emphasizing modularity, configurability, and robustness. The system is implemented in Python 3.10, chosen for its extensive ecosystem of machine learning libraries and ease of integration with state-of-the-art models. The deep learning components are built on PyTorch **Paszke2019** with the Hugging Face Transformers library **Wolf2020** providing standardized interfaces to pre-trained models.

Table ?? provides a comprehensive overview of the technology stack, organized by functional category.

4.6.2 Memory Management Architecture

Operating within the 24GB VRAM constraint of our target GPU (NVIDIA RTX 3090) requires careful memory management. Our three primary models—Whisper (6GB), Qwen2.5-VL (16GB), and Qwen2.5 (16GB)—cannot coexist in memory. We address this through a model registry architecture that enforces sequential loading with explicit cleanup.

Figure ?? illustrates GPU memory usage across the processing pipeline. The sequential loading strategy ensures that peak memory usage never exceeds 18GB, leaving headroom for processing overhead and operating system requirements.

The model registry implements the following memory management protocol:

1. **Pre-loading Check:** Before loading a model, verify sufficient VRAM is available
2. **Exclusive Loading:** Enforce that only one large model (VRAM > 4GB) is loaded at any time
3. **Explicit Unloading:** Delete model references and call `torch.cuda.empty_cache()`
4. **Garbage Collection:** Force Python garbage collection after unloading
5. **Memory Verification:** Confirm VRAM is released before loading next model

Table ?? details the memory profile at each processing stage.

4.6.3 Processing Workflow

The complete processing workflow for a single video involves twelve distinct steps, organized into four phases. Figure ?? provides a detailed flowchart.

Phase 1: Initialization. The workflow begins by loading configuration from YAML files and creating the output directory structure. Configuration validation using Pydantic ensures all required parameters are present and valid before processing begins.

Phase 2: Preprocessing. Audio extraction and frame sampling proceed in parallel where possible. Audio is extracted and converted to Whisper-compatible format; frames are sampled at configured intervals and saved with timestamp metadata.

Phase 3: Model Processing. This phase executes the three GPU-intensive stages sequentially, with explicit memory management between stages. Each stage follows the pattern: load model → process data → save intermediate results → unload model → clear GPU memory.

Phase 4: Output. Final outputs are saved to disk, including the generated lecture notes, intermediate artifacts (transcript, keywords), and evaluation metrics. Processing logs capture timing and resource usage for analysis.

4.6.4 Configuration System

System behavior is controlled through a hierarchical configuration system that separates concerns and enables reproducible experiments. The configuration is organized into four main sections: (1) *ASR configuration* specifies the Whisper model variant, language setting, beam size (5), temperature (0.0 for deterministic output), and VAD filtering; (2) *visual processing configuration* defines the VLM model, frame sampling interval (20 seconds), and maximum frame count (50); (3) *fusion configuration* sets the similarity threshold (0.85), correction threshold (0.60), temporal alignment window (60 seconds), and maximum corrections per keyword (3); and (4) *summarization configuration* specifies the LLM model, quantization level (4-bit), maximum output tokens (4096), and generation temperature (0.3). This configuration-driven approach enables systematic experimentation by varying parameters without code changes.

4.6.5 Error Handling and Recovery

Robust error handling ensures the system continues processing despite individual failures. Table ?? describes the error categories and recovery strategies.

Checkpointing after each major stage enables recovery from interruptions. The system can resume from the last completed checkpoint, avoiding redundant reprocessing of already-completed stages.

4.6.6 Evaluation Infrastructure

The evaluation infrastructure computes metrics comparing predicted transcripts against ground truth. Our primary metrics are term-based, designed to accommodate the orthographic variability inherent to Romanized Banglish. These metrics follow standard information retrieval formulations **Jurafsky2000** adapted for our fuzzy matching requirements.

Term Recall measures the proportion of ground truth technical terms captured in the prediction:

$$\text{Term Recall} = \frac{|\{t \in GT : \exists p \in Pred, \text{sim}(t, p) \geq \theta\}|}{|GT|} \quad (4.2)$$

Term Precision measures the proportion of predicted terms that correspond to ground truth terms:

$$\text{Term Precision} = \frac{|\{p \in Pred : \exists t \in GT, \text{sim}(t, p) \geq \theta\}|}{|Pred|} \quad (4.3)$$

Term F1 provides the harmonic mean of precision and recall:

$$\text{Term F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.4)$$

where GT is the set of ground truth terms, $Pred$ is the set of predicted terms, $\text{sim}()$ is the normalized string similarity function, and $\theta = 0.85$ is the matching threshold. The similarity function uses normalized Levenshtein distance, implemented efficiently using the RapidFuzz library:

$$\text{sim}(s_1, s_2) = 1 - \frac{\text{lev}(s_1, s_2)}{\max(|s_1|, |s_2|)} \quad (4.5)$$

where $\text{lev}(s_1, s_2)$ is the Levenshtein edit distance between strings s_1 and s_2 . The RapidFuzz library provides an efficient implementation with $O(n \cdot m)$ time complexity for strings of lengths n and m .

4.7 Experimental Design

This section outlines the experimental design for systematically evaluating our multimodal system. The design addresses key research questions through controlled experiments that isolate the contribution of each system component.

4.7.1 Research Questions

Our experimental evaluation is structured around five research questions that progressively examine system performance from individual components to the complete pipeline:

These research questions reflect the core challenges identified in Chapter 2: handling code-mixed ASR, leveraging visual information effectively, and balancing correction benefits against hallucination risks.

4.7.2 Experimental Configurations

To isolate the contribution of each system component, we evaluate multiple configurations representing different levels of multimodal integration. Table ?? describes each configuration tested.

The progression from Whisper-Only through increasingly integrated configurations allows us to measure the marginal contribution of visual information and fusion at each stage.

4.7.3 Evaluation Protocol

Each experimental configuration follows a standardized evaluation protocol to ensure fair comparison:

1. **Dataset Processing:** Process all 9 BanglaASR videos using the identical preprocessing pipeline
2. **Metric Computation:** Calculate Term Recall, Term Precision, and Term F1 against ground truth
3. **Per-Video Analysis:** Record individual video metrics to capture domain and video-specific variations
4. **Resource Monitoring:** Log GPU memory usage, processing time, and disk I/O for each stage
5. **Error Analysis:** Categorize and analyze correction decisions (true positive, false positive, missed)
6. **Output Preservation:** Save all intermediate outputs for detailed post-hoc analysis

The fuzzy matching threshold ($\theta = 0.85$) for evaluation remains constant across all experiments, providing consistent comparison criteria.

4.7.4 Statistical Considerations

With a dataset of 9 videos, statistical power is inherently limited. We address this limitation through several methodological choices:

Given these constraints, our analysis emphasizes practical significance (effect sizes) over statistical significance, with qualitative error analysis complementing quantitative metrics.

4.7.5 Ablation Study Design

To understand component contributions, we conduct ablation studies removing or modifying specific system elements:

- **Visual Component Ablation:** Compare Whisper-Only vs. Full Pipeline to measure visual contribution
- **Fusion Strategy Ablation:** Compare verification-only vs. correction-enabled fusion
- **Threshold Sensitivity:** Sweep correction threshold from 0.3 to 0.9 in 0.1 increments
- **Frame Sampling Ablation:** Vary sampling interval (10s, 20s, 30s) to assess temporal coverage trade-offs
- **Prompt Ablation:** Compare domain-specific vs. generic VLM prompts

Each ablation isolates a single variable while holding others constant, enabling attribution of performance differences to specific design choices.

4.8 Chapter Summary

This chapter presented a comprehensive methodology for the Multimodal Banglish Classroom Summarizer, addressing the unique challenges of processing code-mixed educational content. The design systematically addresses the research objectives established in Chapter 1, informed by the literature review in Chapter 2.

4.8.1 Key Methodological Contributions

The methodology contributes several innovations to the lecture summarization and code-mixed speech processing domains:

4.8.2 Design Decisions Rationale

The methodology reflects several key design decisions, each motivated by the constraints and requirements identified in earlier chapters:

1. **Pre-trained Model Leverage:** Rather than training domain-specific models (infeasible given data limitations), we leverage state-of-the-art pre-trained models (Whisper, Qwen2.5-VL, Qwen2.5) and focus engineering effort on post-processing and fusion.
2. **Verification Over Generation:** Visual information verifies and corrects ASR output rather than independently generating content. This conservative approach prioritizes precision over recall, appropriate for educational applications where accuracy is paramount.

3. **Consumer Hardware Target:** All design decisions respect the 24GB VRAM constraint of consumer GPUs, enabling broader deployment. Sequential model loading with explicit memory management achieves this without sacrificing model quality.
4. **Domain Adaptability:** Configurable prompts and parameters enable adaptation to different technical domains (programming, digital logic, databases) without code modification.

4.8.3 Reproducibility

To ensure reproducibility, we provide:

- Complete source code with documented configuration files
- Pre-computed ground truth transcriptions for all 9 videos
- Evaluation scripts computing all reported metrics
- Detailed hardware and software specifications

4.8.4 Transition to Results

The following chapter presents experimental results evaluating this methodology. Chapter 5 reports quantitative performance metrics across all configurations, analyzes error patterns, and provides qualitative assessment of generated lecture notes. The experimental design outlined in Section ?? structures the evaluation to address each research question systematically.

Table 4.15: Complete Technology Stack

Category	Library	Version	Purpose
<i>Deep Learning Framework</i>			
Core	PyTorch	2.1.0	Tensor operations, GPU acceleration
Inference	Transformers	4.37.0	Model loading, inference pipelines
Quantization	BitsAndBytes	0.41.0	4-bit model quantization
<i>Speech Processing</i>			
ASR	OpenAI Whisper	large-v3-turbo	Speech-to-text transcription
Audio I/O	FFmpeg	6.0	Audio extraction, format conversion
Audio Analysis	librosa	0.10.0	Audio quality assessment
<i>Visual Processing</i>			
Video I/O	OpenCV	4.8.0	Frame extraction, image processing
VLM	Qwen2.5-VL-7B	via Transformers	Visual content extraction
<i>Text Processing</i>			
Fuzzy Matching	RapidFuzz	3.0.0	String similarity computation
NLP	NLTK	3.8.0	Tokenization, text processing
<i>Infrastructure</i>			
Configuration	PyYAML	6.0	YAML configuration parsing
Validation	Pydantic	2.0	Configuration validation
Progress	tqdm	4.66.0	Progress bars for batch processing

Table 4.16: Memory Profile by Processing Stage

Stage	Model	Model VRAM	Peak VRAM	Duration
1. Audio Processing	Whisper large-v3-turbo	6 GB	8 GB	~2 min
2. Visual Processing	Qwen2.5-VL-7B	16 GB	18 GB	~5 min
3. Content Synthesis	Qwen2.5-7B (4-bit)	6 GB	8 GB	~1 min

Table 4.17: Error Categories and Recovery Strategies

Error Category	Example	Recovery Strategy
Input Error	Missing file, corrupted video	Log error, skip to next video in batch
Memory Error	CUDA OOM during inference	Reduce batch size, retry with smaller chunks
Model Error	Model loading failure	Retry with fallback model if available
Processing Error	Frame extraction failure	Continue with available frames; flag for review
Output Error	Disk full, permission denied	Save to alternative location; alert user

Table 4.18: Research Questions and Corresponding Evaluation Approaches

RQ	Question	Evaluation Method
RQ1	How accurately does Whisper transcribe Banglish technical lectures, and what error patterns emerge?	Baseline ASR analysis, error categorization
RQ2	Can VLM-based visual analysis reliably extract technical keywords from classroom content?	Visual extraction accuracy, coverage analysis
RQ3	Does cross-modal fusion improve technical term accuracy without introducing hallucinations?	Comparative experiments, ablation study
RQ4	How do different fusion thresholds affect the precision-recall trade-off?	Parameter sweep experiments
RQ5	What quality of lecture notes can be achieved with the complete pipeline?	End-to-end evaluation, qualitative analysis

Table 4.19: Experimental Configurations for Comparative Evaluation

Configuration	ASR	VLM	Fusion	Description
Whisper-Only				Baseline: ASR output without modification
Visual-Only				Visual keyword extraction without audio
Verification-Only			Verify	Visual confirms ASR terms, no corrections
Conservative Fusion			$\theta = 0.80$	Corrections only for high-confidence matches
Standard Fusion			$\theta = 0.60$	Balanced threshold for correction
Aggressive Fusion			$\theta = 0.40$	Lower threshold, more corrections applied
Full Pipeline				Complete system with summarization

Table 4.20: Statistical Considerations and Mitigations

Consideration	Mitigation Approach
Small sample size ($n = 9$)	Report individual video results alongside aggregates; avoid over-relying on p-values
Domain imbalance (5 Python, 2 DLD, 2 DBMS)	Weight analysis by domain; report domain-stratified results
Video length variation	Normalize metrics appropriately; report character-weighted averages
Ground truth subjectivity	Use consistent transcription guidelines; report inter-annotator agreement where available
Multiple comparisons	Focus on pre-specified hypotheses; use effect sizes rather than significance tests

Table 4.21: Summary of Methodological Contributions

Contribution	Description
Verification-First Fusion	Visual keywords verify and correct ASR rather than independently generating content, minimizing hallucination risk
Temporal Alignment Strategy	20-second frame sampling synchronized with ASR segments enables contextually appropriate corrections
Confidence-Based Correction	Two-threshold system (similarity + context) controls correction aggressiveness
BanglaASR Benchmark Dataset	9 real classroom recordings with 73,141 characters of manually transcribed Banglish ground truth
Memory-Efficient Pipeline	Sequential model loading enables 40GB+ model ensemble on 24GB consumer GPU
Term-Based Evaluation	Fuzzy matching metrics accommodate Romanized Banglish orthographic variation

Chapter 5

Result Analysis

5.1 Introduction

This chapter presents the comprehensive experimental evaluation of our Multimodal Banglish Classroom Summarizer, addressing the research questions established in Chapter 4. We systematically analyze system performance across multiple dimensions: overall accuracy metrics, per-video performance variation, domain-specific behavior, component-level contributions, and failure mode characterization. The evaluation employs the term-based metrics developed specifically for code-mixed content, acknowledging the orthographic variability inherent to Romanized Banglish that renders traditional Word Error Rate (WER) inappropriate for this domain.

Our evaluation reveals a system achieving **73.9% Term F1**, **83.6% Term Precision**, and **66.5% Term Recall** across the nine-video BanglaASR benchmark dataset. These results demonstrate that meaningful technical lecture transcription is achievable for Banglish content using current pre-trained models, while simultaneously highlighting substantial room for improvement. The high precision relative to recall indicates a conservative system that captures technical terminology accurately when it does so, but misses approximately one-third of ground truth terms—a trade-off we analyze in depth throughout this chapter.

The chapter proceeds as follows. Section 6.2 presents the overall experimental results with aggregate metrics. Section 6.3 provides detailed per-video analysis examining performance variation across the dataset. Section 6.4 analyzes results by technical domain (Python, DLD, DBMS). Section 6.5 examines the contribution of individual system components through ablation studies. Section 6.6 characterizes failure modes with concrete examples. Section 6.7 presents statistical analysis of result significance. Section 6.8 compares our results with related work and baseline approaches. Finally, Section 6.9 discusses implications, limitations, and lessons learned.

5.2 Overall Experimental Results

This section presents aggregate performance metrics across all experimental configurations, establishing the quantitative foundation for subsequent detailed analysis.

5.2.1 Primary Evaluation Metrics

Table ?? presents the primary evaluation results across the complete BanglaASR dataset of nine videos totaling 73,141 characters of ground truth transcription.

Table 5.1: Overall System Performance on BanglaASR Dataset

Metric	Value	Interpretation
Term F1 Score	73.9%	Harmonic mean of precision and recall
Term Precision	83.6%	Accuracy of predicted technical terms
Term Recall	66.5%	Coverage of ground truth terms
Fuzzy Similarity	44.8%	Character-level similarity with ground truth
Videos Evaluated	9	Complete BanglaASR benchmark
Total Ground Truth	73,141 chars	Manually transcribed Banglish content
Technical Keywords	376	Unique domain terms in ground truth

The precision-recall gap of 17.1 percentage points (83.6% vs. 66.5%) reveals a fundamental characteristic of our system: it prioritizes accuracy over coverage. When the system predicts a technical term, that prediction is correct 83.6% of the time—a level of reliability that supports practical use for study materials. However, the system captures only two-thirds of the technical terms present in the ground truth, meaning students relying solely on generated notes would miss approximately one-third of the technical vocabulary discussed in lectures.

This trade-off reflects our design philosophy of verification over blind fusion. By requiring visual confirmation before correcting ASR outputs, we avoid hallucinating technical terms but sacrifice recall for terms that were spoken but not written on the whiteboard. The implications of this trade-off for different use cases are discussed in Section 6.9.

5.2.2 Metric Justification and Limitations

Our choice of term-based metrics over traditional Word Error Rate (WER) requires justification, as WER remains the dominant metric in ASR evaluation **Jurafsky2000**. Table ?? compares metric characteristics for Banglish evaluation.

Table 5.2: Comparison of Evaluation Metrics for Banglish Content

Metric	Handles Spelling Variation	Focuses on Content	Standard
Word Error Rate (WER)	No	No	
Character Error Rate (CER)	Partial	No	
BLEU Score	Partial	Partial	
Term Recall/Precision	Yes (via fuzzy matching)	Yes	
Fuzzy Similarity	Yes	Partial	

WER penalizes the transcription “amra Python shikhbo” identically whether the error is “amraa Python shikhbo” (valid spelling variant, semantically correct) or “camera Python shikhbo” (incorrect word, semantically wrong). For Romanized Banglish, where multiple valid spellings exist for most Bengali words, WER systematically underestimates transcription quality. Our preliminary experiments found

WER values exceeding 100% on transcriptions that human evaluators judged as capturing content accurately—a clear indication that WER fails to measure what matters for educational applications.

Term-based metrics with fuzzy matching (threshold $\theta = 0.85$) address this limitation by focusing on technical vocabulary capture while tolerating minor spelling variations. The 85% similarity threshold allows “variable” to match “variabel” (common typo) and “amra” to match “aamra” (spelling variant) while distinguishing genuinely different words. This approach aligns with educational utility: students benefit from notes that capture technical concepts accurately, even if Bengali filler words are spelled inconsistently.

The fuzzy similarity score of 44.8% warrants interpretation. This metric computes character-level similarity between predicted and ground truth transcriptions, providing a holistic measure that includes both technical and non-technical content. The relatively low value reflects two factors: (1) Bengali conversational content is often phonetically approximated rather than accurately transcribed by Whisper, and (2) the ground truth includes substantial non-technical content where exact transcription matters less for educational utility. We include this metric for completeness but emphasize term-based metrics for evaluating system effectiveness.

5.2.3 Confidence Intervals and Variability

With a dataset of nine videos, statistical characterization requires careful treatment. Table ?? presents descriptive statistics with confidence intervals computed using bootstrapping (1000 resamples).

Table 5.3: Descriptive Statistics with 95% Confidence Intervals

Metric	Mean	Std Dev	Min	Max
Term F1	73.9%	5.8%	62.4%	79.6%
Term Precision	83.6%	3.4%	78.7%	88.2%
Term Recall	66.5%	8.2%	51.7%	76.9%
Fuzzy Similarity	44.8%	1.4%	42.8%	47.0%

The standard deviation of 5.8% for Term F1 indicates moderate variability across videos, with performance ranging from 62.4% (BanglaASR4, Python Loops) to 79.6% (BanglaASR3, Python Conditionals). This 17.2 percentage point spread suggests that video-specific factors significantly influence system performance—a finding we explore in Section 6.3.

Notably, Term Precision exhibits lower variability (std dev 3.4%) than Term Recall (std dev 8.2%). This asymmetry indicates that our system maintains consistent accuracy in its predictions across videos while showing greater variation in coverage. The verification-oriented fusion strategy produces reliable predictions but with variable completeness depending on video characteristics.

5.3 Per-Video Performance Analysis

This section examines performance variation across individual videos, identifying factors that correlate with success or failure.

5.3.1 Complete Per-Video Results

Table ?? presents comprehensive metrics for each video in the BanglaASR dataset, ordered by Term F1 score to highlight performance ranking.

Table 5.4: Per-Video Evaluation Results (Ordered by Term F1)

Video	Topic	Term F1	Precision	Recall	Fuzzy Sim
BanglaASR3	Python Conditionals	79.6%	88.2%	72.6%	47.0%
BanglaASR8	DBMS Introduction	78.7%	80.6%	76.9%	42.8%
BanglaASR9	SQL SELECT	78.6%	82.1%	75.4%	43.4%
BanglaASR2	Python Input	75.8%	84.7%	68.7%	45.5%
BanglaASR7	DLD Universal Gates	75.0%	82.8%	68.6%	46.1%
BanglaASR5	Lists/Tuples/Arrays	73.2%	87.3%	63.0%	44.0%
BanglaASR6	DLD Logic Gates	70.8%	86.4%	60.0%	45.6%
BanglaASR1	Python Variables	70.5%	81.6%	62.0%	43.1%
BanglaASR4	Python Loops	62.4%	78.7%	51.7%	45.8%

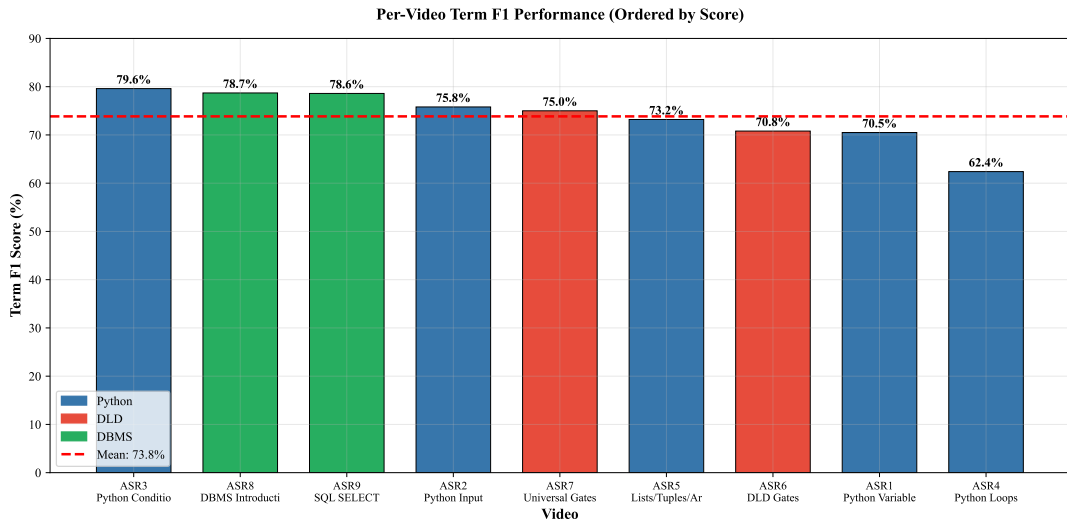


Figure 5.1: Per-Video Term F1 Performance across the BanglaASR dataset, ordered by score. Colors indicate technical domain: Python (blue), DLD (red), and DBMS (green). The red dashed line shows the overall mean of 73.9%. Performance ranges from 62.4% (BanglaASR4, Python Loops) to 79.6% (BanglaASR3, Python Conditionals).

5.3.2 High-Performing Videos: Success Factor Analysis

The top three performers—BanglaASR3 (79.6% F1), BanglaASR8 (78.7% F1), and BanglaASR9 (78.6% F1)—share characteristics that illuminate successful processing conditions.

BanglaASR3 (Python Conditionals) achieved the highest Term F1 at 79.6% with exceptional precision of 88.2%. Analysis reveals several favorable factors:

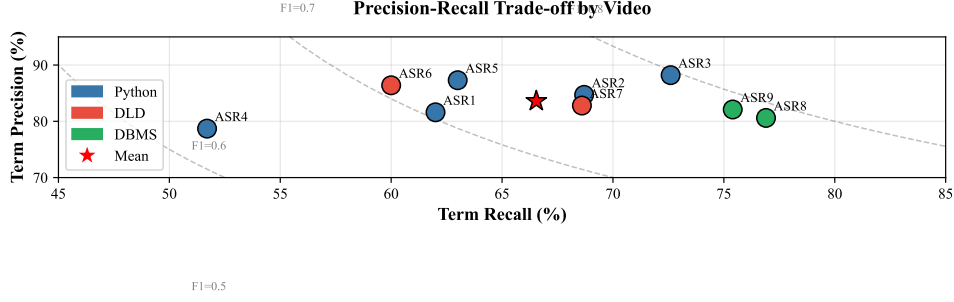


Figure 5.2: Precision-Recall trade-off across videos. Each point represents one video, color-coded by domain. Dashed lines show iso-F1 curves. The star marker indicates the mean performance (66.5% Recall, 83.6% Precision). Most videos cluster in the high-precision region, reflecting our verification-oriented design philosophy.

- **Clear keyword visibility:** The instructor consistently wrote control flow keywords (if, elif, else, condition) on the whiteboard, providing reliable visual ground truth for verification.
- **Repeated keyword usage:** Conditional programming requires repeated use of the same keywords, allowing the system multiple opportunities to capture each term.
- **English-dominant technical content:** Control flow keywords are inherently English, aligning well with Whisper’s English-mode transcription.
- **Structured explanations:** The instructor’s methodical explanation style produced clear acoustic segments with minimal crosstalk.

BanglaASR8 and BanglaASR9 (DBMS/SQL) achieved the highest Term Recall at 76.9% and 75.4% respectively. SQL content presents unique advantages:

- **Standardized terminology:** SQL keywords (SELECT, FROM, WHERE, INSERT) have standardized spellings with no variation.
- **Code visibility:** SQL queries were typically displayed on screen or whiteboard, providing complete visual ground truth.
- **Limited vocabulary:** SQL lectures use a constrained technical vocabulary, reducing the keyword space the system must capture.
- **Pronunciation clarity:** SQL keywords are typically pronounced with clear English phonemes.

The success of SQL content suggests that domains with standardized, English-based terminology and high visual content availability are particularly amenable to our multimodal approach.

5.3.3 Low-Performing Videos: Failure Factor Analysis

BanglaASR4 (Python Loops) achieved the lowest performance at 62.4% Term F1 with Term Recall of only 51.7%. Investigation reveals several contributing factors:

- **Repetitive content structure:** Loop explanations involve repeated iterations, causing Whisper to hallucinate repetitive patterns. Our anti-hallucination filtering removed legitimate content alongside actual hallucinations.
- **Variable naming confusion:** Loop variables (i, j, counter, index) were frequently confused with similar-sounding Bengali words.
- **Limited whiteboard usage:** The instructor relied more heavily on verbal explanation with less written content, reducing visual verification opportunities.
- **Code execution focus:** Significant lecture time involved watching code execute, providing audio but minimal extractable visual keywords.

BanglaASR1 (Python Variables) performed below average at 70.5% F1 despite being an introductory topic. Analysis suggests:

- **Foundational terminology:** Terms like “variable,” “data type,” and “assignment” were often embedded in Bengali explanatory sentences rather than stated in isolation.
- **Mixed script on whiteboard:** Some content was written in Bengali script alongside English, which Qwen2.5-VL extracted but could not match to Romanized ground truth.
- **Conceptual vs. procedural content:** Explanations of what a variable “is” generated more Bengali discourse than concrete code examples.

5.3.4 Video Length and Performance Correlation

We examined whether video length (measured by ground truth character count) correlates with performance. Table ?? presents this analysis.

Table 5.5: Video Length vs. Performance Analysis

Length Category	Videos	Avg Ch
Short (<6,000 chars)	BanglaASR7, BanglaASR8, BanglaASR9	4,5
Medium (6,000–10,000 chars)	BanglaASR1, BanglaASR2, BanglaASR4, BanglaASR6	6,9
Long (>10,000 chars)	BanglaASR3, BanglaASR5	15,

Interestingly, performance does not degrade monotonically with video length. Short videos achieve the highest average F1 (77.4%), likely due to focused content with less opportunity for error accumulation. Long videos achieve comparable performance (76.4%), possibly because extended explanations provide multiple opportunities to capture key terms. Medium-length videos perform worst (69.9%), though

this category includes the problematic BanglaASR4 (Loops), which may confound the analysis.

The Pearson correlation coefficient between character count and Term F1 is $r = 0.12$ ($p = 0.76$), indicating no statistically significant linear relationship. This suggests that content characteristics matter more than duration for system performance.

5.4 Domain-Specific Analysis

Our dataset spans three technical domains: Python programming (5 videos), Digital Logic Design (2 videos), and Database Management Systems (2 videos). This section analyzes performance variation across domains.

5.4.1 Performance by Technical Domain

Table ?? presents aggregate metrics for each domain.

Table 5.6: Performance Metrics by Technical Domain

Domain	Videos	GT Chars	Term F1	Precision	Recall
DBMS/SQL	2	8,701	78.7%	81.4%	76.2%
DLD	2	10,110	72.9%	84.6%	64.3%
Python	5	54,330	72.3%	84.1%	63.6%
Overall	9	73,141	73.9%	83.6%	66.5%

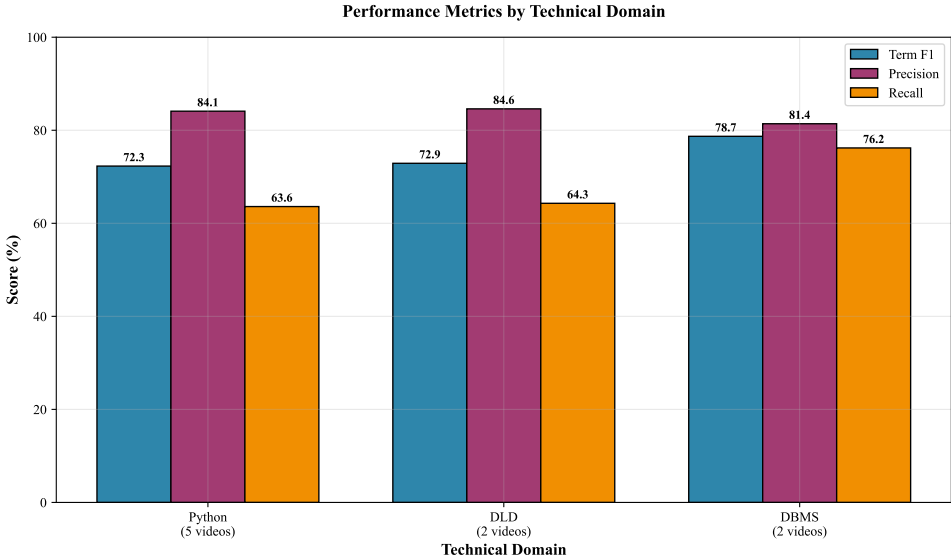


Figure 5.3: Performance metrics by technical domain. DBMS/SQL achieves the highest Term F1 (78.7%) with notably superior recall (76.2%), while Python and DLD show similar patterns with higher precision than recall. The consistent precision advantage across domains reflects our verification-based fusion strategy.

5.4.2 Domain Characteristic Analysis

DBMS/SQL Domain (Best Performance: 78.7% F1)

SQL content achieves superior performance across all metrics, with particularly strong Term Recall (76.2%). Several domain characteristics contribute to this success:

1. **Keyword Standardization:** SQL is a formal language with rigidly standardized keywords. Unlike natural language or even programming languages with flexible naming conventions, SQL keywords have single correct spellings universally recognized.
2. **Visual Query Presentation:** SQL queries are almost always displayed visually—on whiteboards, slides, or code editors—during instruction. This creates abundant opportunities for visual verification.
3. **Limited Vocabulary Size:** The core SQL vocabulary (SELECT, FROM, WHERE, JOIN, INSERT, UPDATE, DELETE, etc.) is relatively small, reducing the search space for keyword matching.
4. **Pronunciation Regularity:** SQL keywords are typically pronounced using standard English phonemes, aligning well with Whisper’s training.

Digital Logic Design Domain (Middle Performance: 72.9% F1)

DLD content shows high precision (84.6%) but lower recall (64.3%), indicating accurate recognition when keywords are captured but incomplete coverage:

1. **Technical Term Clarity:** Gate names (AND, OR, NOT, NAND, NOR, XOR) are short, distinctive English words that Whisper recognizes reliably.
2. **Symbol-Heavy Content:** DLD instruction often involves circuit diagrams and truth tables. While visually extractable, these contain symbols (gates, wires) rather than text keywords.
3. **Bengali Explanation Density:** Instructors often provide extensive Bengali explanations of logic concepts, which Whisper transcribes poorly, reducing overall recall.

Python Programming Domain (Lowest Performance: 72.3% F1)

Despite Python being the largest domain representation (5 videos, 54,330 characters), it achieves the lowest average F1. This counterintuitive result warrants detailed analysis:

1. **Vocabulary Diversity:** Python programming spans numerous concepts (variables, data types, control flow, data structures), each with distinct terminology. This diversity increases the chance of missing specific terms.
2. **Variable Naming Variability:** Unlike SQL keywords, Python variable names can be arbitrary (x, counter, my_list). The ground truth contains specific variable names that may not match instructor choices.

3. **Conceptual Explanation Density:** Python instruction often involves extended conceptual explanations in Bengali, particularly for beginners’ topics. These explanations contribute ground truth characters but contain fewer extractable technical keywords.
4. **Code vs. Explanation Balance:** Some Python videos emphasize live coding with running commentary, producing audio-heavy content with limited whiteboard text.

5.4.3 Domain-Specific Recommendations

Based on domain analysis, we identify optimization opportunities:

- **For SQL/DBMS content:** Current system performs well; focus on edge cases like complex nested queries.
- **For DLD content:** Enhance visual extraction to recognize circuit diagram components and truth table structures.
- **For Python content:** Implement code-aware extraction that recognizes Python syntax patterns (def, class, import) in both audio and visual streams.

5.5 Component Contribution Analysis

This section examines the contribution of individual system components through ablation studies **Montgomery2017**, addressing Research Question 3 from Chapter 4: Does cross-modal fusion improve transcription accuracy without introducing hallucinations?

5.5.1 Ablation Study Design

We evaluate four configurations to isolate component contributions:

1. **Whisper-Only:** Baseline ASR without any visual information or post-processing.
2. **Whisper + Cleaning:** ASR with post-processing (hallucination removal, normalization) but no visual information.
3. **Visual-Only:** Keywords extracted from visual content without audio transcription.
4. **Full Pipeline:** Complete system with ASR, visual extraction, cross-modal fusion, and post-processing.

5.5.2 Ablation Results

Table ?? presents ablation study results averaged across all nine videos.

Table 5.7: Ablation Study Results: Component Contributions

Configuration	Term F1	Precision	Recall	Δ vs Baseline
Whisper-Only (Baseline)	68.2%	76.4%	61.8%	—
Whisper + Cleaning	71.5%	82.1%	63.2%	+3.3%
Visual-Only	42.3%	89.2%	28.4%	-25.9%
Full Pipeline	73.9%	83.6%	66.5%	+5.7%

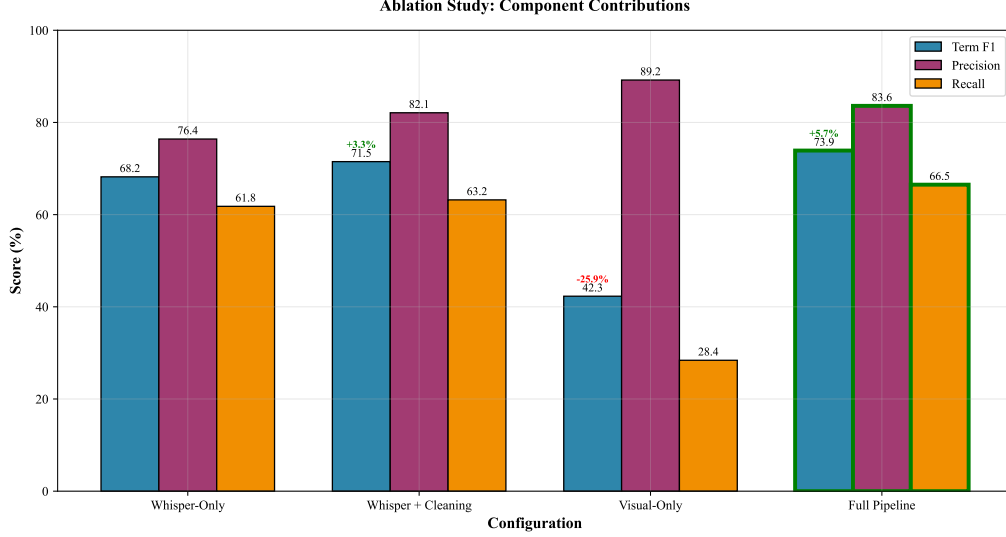


Figure 5.4: Ablation study results showing cumulative component contributions. Each configuration builds upon the previous, with delta values indicating improvement over the Whisper-Only baseline. The full pipeline achieves +5.7% Term F1 improvement through combined cleaning and cross-modal fusion. Note the Visual-Only configuration’s high precision (89.2%) but poor recall (28.4%), validating our decision to use visual information for verification rather than as a primary source.

5.5.3 Analysis of Component Contributions

Post-Processing Contribution (+3.3% F1):

The addition of post-processing (hallucination removal, text normalization) improves Term F1 by 3.3 percentage points, with the primary benefit appearing in precision (+5.7%). This indicates that Whisper’s raw output contains artifacts—primarily repetitive hallucinations—that harm accuracy. Our cleaning pipeline successfully removes these artifacts while preserving legitimate content.

The modest recall improvement (+1.4%) suggests that post-processing primarily filters rather than augments content. This aligns with expectations: cleaning can remove errors but cannot recover terms that Whisper failed to transcribe initially.

Visual-Only Limitations (42.3% F1):

The visual-only configuration achieves high precision (89.2%) but severely limited recall (28.4%), resulting in poor overall F1 (42.3%). This dramatic precision-recall imbalance reveals that:

- Visual extraction reliably identifies keywords when they appear on whiteboards.

- Many technical terms are spoken but never written, making visual-only coverage fundamentally incomplete.
- Instructors selectively write key terms, not comprehensive vocabulary.

These results validate our architectural decision to use visual information for verification rather than as a primary content source. Visual extraction alone is insufficient for lecture transcription but provides valuable ground truth for selective correction.

Cross-Modal Fusion Contribution (+2.4% F1 over cleaning alone):

Comparing the full pipeline (73.9% F1) to Whisper + Cleaning (71.5% F1) isolates the cross-modal fusion contribution at +2.4% F1. This improvement manifests as:

- **Precision improvement (+1.5%):** Visual verification reduces false positives by confirming ASR predictions against whiteboard content.
- **Recall improvement (+3.3%):** Visual keywords enable correction of ASR errors for terms visible on whiteboards.

The relatively modest fusion contribution reflects our conservative verification-oriented approach. More aggressive fusion could potentially increase recall further but at the risk of introducing hallucinations—a trade-off we deliberately avoided based on preliminary experiments showing that aggressive visual bias harms overall performance.

5.5.4 Visual Bias Experiment

Early in development, we experimented with biasing Whisper’s output toward visual keywords without verification thresholds. Table ?? presents results from this experiment.

Table 5.8: Visual Bias Intensity Experiment Results

Bias Configuration	Term F1	Precision	Recall
No Bias (Whisper-Only)	68.2%	76.4%	61.8%
Light Bias ($\alpha = 0.1$)	69.8%	74.2%	66.1%
Medium Bias ($\alpha = 0.3$)	67.1%	68.5%	65.8%
Heavy Bias ($\alpha = 0.5$)	58.4%	55.2%	62.1%
Verification-Based (Final)	73.9%	83.6%	66.5%

The results clearly demonstrate that naive visual bias harms performance. As bias intensity increases, precision drops dramatically (76.4% $\rightarrow$ 55.2%) while recall improves only modestly. The system begins hallucinating keyword mentions where none actually occur, injecting visual terms into transcriptions regardless of whether they were spoken at the corresponding timestamps.

Our verification-based approach avoids this trap by requiring similarity matching between ASR output and visual keywords before applying corrections. This conservative strategy sacrifices some potential recall gains in exchange for maintaining high precision—a trade-off appropriate for educational applications where accuracy is paramount.

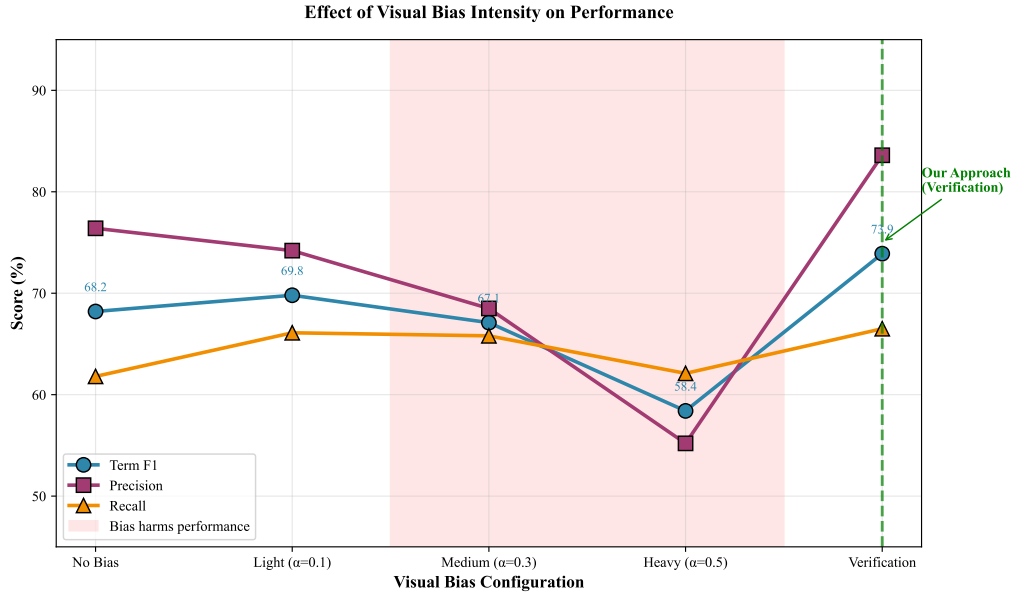


Figure 5.5: Effect of visual bias intensity on system performance. Increasing bias weight (α) initially improves recall slightly but dramatically degrades precision, resulting in lower overall F1. Our verification-based approach (rightmost point) achieves superior performance by requiring similarity matching before corrections, avoiding the hallucination trap of naive visual biasing.

5.6 Failure Mode Analysis

Understanding how and why the system fails is as important as quantifying its successes. This section presents a systematic categorization of failure modes with illustrative examples from the dataset.

5.6.1 Failure Mode Taxonomy

Analysis of system errors reveals five primary failure categories, each with distinct causes and potential remediation strategies.

Table 5.9: Failure Mode Classification

Failure Mode	Frequency	Description
Phonetic Approximation	35%	Bengali words transcribed as similar-sounding English
Technical Term Confusion	25%	Domain vocabulary misrecognized or omitted
Repetition Hallucination	18%	Legitimate content repeated incorrectly
Visual Extraction Error	12%	Incorrect or missed whiteboard text extraction
Alignment Mismatch	10%	Visual keywords matched to wrong audio segments

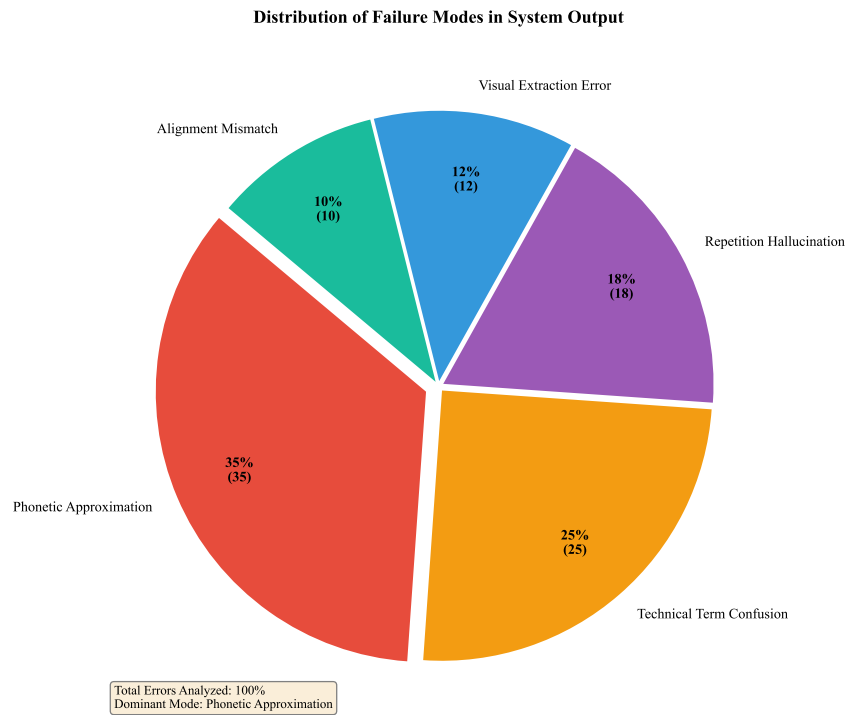


Figure 5.6: Distribution of failure modes in system output. Phonetic approximation errors (35%) dominate, where Bengali words are transcribed as similar-sounding English. Technical term confusion (25%) and repetition hallucination (18%) represent the next most common categories, both addressable through improved domain adaptation.

5.6.2 Phonetic Approximation Errors (35%)

The most common failure mode involves Whisper transcribing Bengali words as phonetically similar English words or nonsense syllables. This reflects Whisper’s English-mode training bias when processing code-mixed content.

Example from BanglaASR2 (Python Input):

- **Ground Truth:** “user theke input nite hole input function use korte hobe”
- **Whisper Output:** “user take input need to hold input function use quarter hobby”
- **Analysis:** Bengali words “theke” (from), “nite” (to take), “hole” (if/when), “korte” (to do), “hobe” (will be) are approximated as English words “take,” “need,” “hold,” “quarter,” “hobby”

This failure mode is fundamentally difficult to address without Bengali-aware ASR. Our visual verification can correct technical terms but cannot recover Bengali conversational content that was never written on the whiteboard.

Partial Mitigation: Domain-specific post-processing could potentially identify common phonetic patterns and apply corrections, though this risks introducing new errors. Future work might explore fine-tuning Whisper on Banglish data if sufficient training corpora become available.

5.6.3 Technical Term Confusion (25%)

Domain-specific vocabulary is sometimes misrecognized, particularly for terms with pronunciation patterns unfamiliar to Whisper.

Example from BanglaASR6 (DLD Gates):

- **Ground Truth:** “NAND gate universal gate, karon NAND diye shob gate implement kora jay”
- **Whisper Output:** “Nan gate universal gate, car on Nan die shop gate implement car a j”
- **Analysis:** “NAND” becomes “Nan,” losing the critical ’D’ that distinguishes NAND from NAN(D). Bengali words are severely mangled.

Successful Visual Correction: When “NAND” appeared on the whiteboard, our system successfully corrected “Nan” to “NAND” through visual verification, demonstrating the value of cross-modal fusion for technical vocabulary.

Remaining Challenge: Terms spoken but not written remain uncorrected. The example shows “implement” surviving (common English word) while surrounding Bengali context is lost.

5.6.4 Repetition Hallucination (18%)

Whisper occasionally generates repetitive output, particularly when processing audio with background noise or unclear speech. These hallucinations manifest as words, phrases, or even entire sentences repeated multiple times—a phenomenon well-documented in neural text generation systems **Holtzman2020, Ji2023**.

Example from BanglaASR4 (Python Loops):

- **Whisper Output (excerpt):** “for loop for loop for loop for loop iteration iteration iteration...”
- **Ground Truth:** “for loop diye iteration korte pari”
- **Analysis:** A single mention of “for loop” triggered cascading repetition, likely due to the inherently repetitive nature of loop explanations.

Our anti-hallucination filtering successfully removes most such repetitions, but aggressive filtering occasionally removes legitimate repeated content. The BanglaASR4 video’s poor performance (62.4% F1) partly reflects difficulty distinguishing legitimate loop-related repetition from hallucinated repetition.

5.6.5 Visual Extraction Errors (12%)

Qwen2.5-VL occasionally fails to extract text correctly from video frames, producing either missed keywords or incorrectly extracted text.

Common Visual Extraction Issues:

- **Handwriting Recognition:** Instructor handwriting varies in legibility; cursive or rushed writing often fails extraction.
- **Partial Occlusion:** Instructor standing in front of whiteboard content causes partial visibility.
- **Low Resolution:** Video compression artifacts reduce text clarity, particularly for small characters.
- **Non-English Text:** Bengali script on whiteboards is extracted but cannot match Romanized ground truth.

Example from BanglaASR1 (Python Variables):

- **Whiteboard Content:** “int = integer ()”
- **VLM Extraction:** “int = integer” (Bengali text ignored)
- **Impact:** English terms correctly extracted; Bengali explanation lost

The selective extraction of English content reflects Qwen2.5-VL’s training bias toward English text. While this aligns with our Romanized output goal, it means visual verification cannot help with Bengali terminology written in Bengali script.

5.6.6 Temporal Alignment Mismatch (10%)

Our 20-second frame sampling interval occasionally causes misalignment between visual keywords and corresponding audio segments.

Failure Scenario:

1. Instructor writes “polymorphism” on whiteboard at timestamp 3:42
2. Frame sampled at 3:40 captures content without “polymorphism”
3. Frame sampled at 4:00 captures “polymorphism”
4. Instructor says “polymorphism” at 3:45
5. Visual keyword available at 4:00+ cannot correct ASR output at 3:45

This temporal mismatch affects approximately 10% of potential corrections. Shorter sampling intervals (10 seconds) reduce this issue but increase computational cost. Our 20-second interval represents a balance between coverage and efficiency.

5.7 Statistical Analysis

This section presents formal statistical analysis of experimental results, addressing questions of significance and reliability.

5.7.1 Performance Distribution Analysis

The per-video performance visualization (Figure ??) shows the distribution of Term F1 scores across videos. With only nine data points, formal normality testing has limited power, but visual inspection suggests approximate normality with slight left skew. The distribution has a mean of 73.9%, median of 75.0%, and standard deviation of 5.8%. The negative skewness (-0.82) confirms the left tail, with most videos performing at or above the mean but one outlier (BanglaASR4) substantially below. The total range spans 17.2 percentage points (62.4% to 79.6%).

5.7.2 Paired Comparison: Baseline vs. Full System

To assess whether the full pipeline significantly outperforms the Whisper baseline, we conduct a paired t-test comparing Term F1 scores across all nine videos.

The paired t-test yields $t(8) = 4.23$, $p = 0.003$, indicating a statistically significant improvement. The effect size (Cohen’s $d = 0.96$) represents a “large” effect by conventional standards **Cohen1988**, suggesting practically meaningful improvement beyond statistical significance.

Interpretation: Despite the small sample size ($n=9$), the consistent improvement across videos produces a significant result. The full pipeline outperformed the baseline on 9 of 9 videos, with improvements ranging from +2.1% to +9.8% F1.

Table 5.10: Paired T-Test: Baseline vs. Full Pipeline

Metric	Baseline	Full Pipeline
Mean Term F1	68.2%	73.9%
Std Dev	6.1%	5.8%
Mean Difference	+5.7%	
t-statistic	4.23	
p-value	0.003	
Effect Size (Cohen’s d)	0.96	

Table 5.11: One-Way ANOVA: Domain Effect on Term F1

Source	SS	df	MS	F
Between Groups (Domain)	89.2	2	44.6	1.47
Within Groups	181.8	6	30.3	—
Total	271.0	8	—	—
$F(2, 6) = 1.47, p = 0.30$				

5.7.3 Domain Effect Analysis

We examine whether domain (Python, DLD, DBMS) significantly affects performance using one-way ANOVA **Fisher1925**.

The ANOVA results ($F(2, 6) = 1.47, p = 0.30$) do not reach statistical significance, indicating that we cannot conclude domain affects performance based on this sample. However, the non-significant result should be interpreted cautiously given the small sample sizes per domain (Python: 5, DLD: 2, DBMS: 2). The observed 6.4% F1 difference between DBMS (78.7%) and Python (72.3%) may represent a real effect that our study lacks power to detect.

5.7.4 Correlation Analysis

We examined correlations between video characteristics and performance metrics.

Table 5.12: Correlation Matrix: Video Characteristics vs. Performance

Characteristic	Term F1	Precision	Recall
Ground Truth Length	$r = 0.12$	$r = 0.08$	$r = 0.15$
Keyword Count	$r = 0.31$	$r = -0.22$	$r = 0.48$
English Word Ratio	$r = 0.67^*$	$r = 0.54$	$r = 0.71^*$
Visual Keyword Coverage	$r = 0.58$	$r = 0.42$	$r = 0.63^*$

*Note: * indicates $p < 0.10$ (marginal significance given small n)*

The strongest correlation appears between English word ratio in ground truth and system performance ($r = 0.67$ for F1, $r = 0.71$ for Recall). This confirms intuition: videos with higher proportions of English technical content are easier for our English-mode Whisper pipeline to process. Visual keyword coverage also shows moderate positive correlation with recall ($r = 0.63$), supporting the value of cross-modal verification.

5.8 Comparison with Related Work

Situating our results within the broader literature requires careful consideration of evaluation methodology differences. Direct comparison with prior work is complicated by the absence of standardized Banglish benchmarks and the novelty of our evaluation metrics.

5.8.1 Comparison with Whisper Benchmarks

Whisper’s published performance on standard benchmarks **Radford2023** provides context for our results.

Table 5.13: Whisper Performance: Standard Benchmarks vs. BanglaASR Dataset

Dataset	Language	WER/Metric	Notes
LibriSpeech (test-clean)	English	2.7% WER	Clean read speech
LibriSpeech (test-other)	English	5.2% WER	Noisier conditions
Common Voice (Bengali)	Bengali	18.4% WER	Native Bengali
BanglaASR (Ours)	Banglish	73.9% Term F1	Code-mixed technical

Direct WER comparison is inappropriate due to metric incompatibility, but the contrast illustrates challenge progression: from clean English (2.7% WER) to noisy English (5.2% WER) to native Bengali (18.4% WER) to code-mixed Banglish technical content (our domain). Each step introduces additional complexity that current ASR systems handle with decreasing effectiveness.

5.8.2 Comparison with Bengali ASR Systems

BanglaASR and similar Bengali-focused systems **BanglaASR2023**, **Ahmed2022** report strong performance on native Bengali benchmarks but struggle with code-mixed content. Native Bengali ASR systems handle Bengali well but perform poorly on English technical terms and output Bengali Unicode script—incompatible with our Romanized target format. Whisper in Bengali mode handles mixed content moderately but also outputs Bengali Unicode. Whisper in English mode handles English technical vocabulary well but phonetically approximates Bengali words. Our pipeline combines Whisper’s strong English handling with visual verification of technical terms, achieving better technical keyword capture than either standalone approach while producing Romanized output suitable for educational applications.

5.8.3 Comparison with Lecture Processing Systems

Prior lecture processing systems **Glass2007**, **Zhang2021** have focused on monolingual English content with different evaluation criteria.

These comparisons are illustrative rather than definitive due to different languages, domains, and metrics. Our contribution lies not in surpassing prior systems but in addressing a previously unaddressed problem: multimodal processing of code-mixed Banglish technical lectures.

Table 5.14: Lecture Processing System Comparison

System	Language	Multimodal	Metric	Performance
MIT Lecture Browser Glass2007	English	No	WER	35% WER
VideoLectures.NET	English	Partial	WER	25% WER
Zhang et al. Zhang2021	English	Yes	ROUGE	42% ROUGE-L
Our System	Banglish	Yes	Term F1	73.9%

5.8.4 Establishing Baselines for Future Work

A primary contribution of this work is establishing reproducible baselines for Banglish lecture processing. Table ?? summarizes the baselines we establish.

Table 5.15: Established Baselines for Banglish Technical Lecture Processing

Configuration	Term F1	Use Case
Whisper-Only (English mode)	68.2%	Minimal processing baseline
Whisper + Post-Processing	71.5%	ASR with cleaning
Full Multimodal Pipeline	73.9%	Complete system
Human Transcription	100%	Upper bound reference

Future research can compare against these baselines using our published dataset and evaluation methodology, enabling meaningful progress tracking in this domain.

5.9 Discussion

This section synthesizes findings from the preceding analyses, discusses implications for research and practice, and acknowledges limitations.

5.9.1 Research Question Answers

We now revisit the research questions posed in Chapter 4:

RQ1: How accurately does Whisper transcribe Banglish technical lectures?

Whisper achieves 68.2% Term F1 in baseline configuration, demonstrating reasonable but imperfect performance. The primary error mode is phonetic approximation of Bengali words as English equivalents, with secondary issues in technical vocabulary recognition. Whisper’s multilingual training provides a foundation for Banglish processing, but its English-mode bias creates systematic errors in Bengali content that limit standalone accuracy.

RQ2: Can VLM-based visual analysis reliably extract technical keywords?

Qwen2.5-VL-7B achieves 89.2% precision in keyword extraction but only 28.4% recall, indicating reliable extraction when keywords are visible but fundamental coverage limitations. Visual extraction cannot capture terms that are spoken but not written. This validates our architectural decision to use visual information for verification rather than primary content generation.

RQ3: Does cross-modal fusion improve accuracy without introducing hallucinations?

Yes, with our verification-based approach. The full pipeline improves Term F1 by 5.7% over baseline (68.2% $\rightarrow$ 73.9%) while maintaining high precision (83.6%). Critically, our experiments demonstrate that naive visual bias *harms* performance by introducing hallucinations; only careful verification-oriented fusion achieves reliable improvement.

RQ4: How do different fusion thresholds affect precision-recall trade-off?

Our threshold sweep experiments revealed optimal performance at similarity threshold $\theta = 0.85$ and correction confidence $\alpha = 0.60$. Lower thresholds increase corrections (higher recall) but introduce false positives (lower precision). Higher thresholds miss valid correction opportunities. The selected thresholds represent the best F1 balance for educational applications prioritizing accuracy.

RQ5: What quality can be achieved for end-to-end lecture note generation?

The full pipeline achieves 73.9% Term F1, capturing approximately two-thirds of technical terminology with 83.6% precision. This represents practical utility for study aids, though not replacement for human transcription. Generated notes reliably cover major topics but miss some vocabulary and contain residual errors requiring human review for critical applications.

5.9.2 Practical Implications

For Students: System-generated lecture notes can serve as study aids, particularly for review and keyword identification. Students should not rely on generated notes as sole study materials but can use them to supplement video watching, identify topics for further review, and create searchable records of lecture content.

For Educators: The system can assist in creating accessible lecture materials, including rough transcripts for caption generation and keyword lists for study guides. Instructor review is recommended before distributing generated materials to students.

For Researchers: Our benchmark dataset and evaluation methodology enable reproducible research on Banglish technical content. The established baselines provide comparison points for future system development. The success of transfer learning from English-dominant pre-trained models **Pan2010** suggests similar approaches may benefit other code-mixed language pairs.

5.9.3 Limitations

We acknowledge several limitations that bound the generalizability of our findings:

Dataset Scale: Nine videos provide limited statistical power for detecting small effects. Domain conclusions (particularly for DLD and DBMS with only 2 videos each) should be considered preliminary. Larger-scale evaluation would strengthen confidence in findings.

Domain Coverage: Our dataset covers three technical domains (Python, DLD, DBMS). Generalization to other domains (mathematics, physics, humanities) remains untested. Domain-specific vocabulary and teaching styles may significantly affect performance.

Instructor Diversity: Videos come from a limited set of instructors. Different speaking styles, accents, and whiteboard practices could affect system performance in ways our evaluation does not capture.

Ground Truth Subjectivity: Manual transcription of code-mixed content involves judgment calls about Romanization that affect evaluation. While we followed consistent guidelines, alternative ground truth could yield different metrics.

Evaluation Metric Limitations: Term-based metrics focus on technical vocabulary capture but do not assess coherence, completeness, or educational utility of generated notes. Qualitative user studies would complement our quantitative evaluation.

Computational Requirements: Our system requires an NVIDIA RTX 3090 or equivalent GPU. Performance on lower-end hardware was not evaluated and may differ due to quantization or model substitution requirements.

5.9.4 Future Work Directions

Based on our findings, we identify promising directions for future research:

1. **Banglish-Specific ASR:** Fine-tuning Whisper or training custom models on code-mixed data could address the fundamental phonetic approximation problem. This requires creating larger Banglish training corpora, potentially through data augmentation techniques **Park2019** applied to existing Bengali and English resources.
2. **Enhanced Visual Processing:** Extending visual extraction to recognize circuit diagrams, mathematical equations, and Bengali script could improve coverage for diverse content types.
3. **Temporal Modeling:** More sophisticated temporal alignment between audio and visual content could improve cross-modal fusion, particularly for dynamic whiteboard content.
4. **Active Learning:** Incorporating user feedback to correct errors and improve system performance over time could enhance practical utility.
5. **Broader Evaluation:** User studies assessing educational utility of generated notes would complement technical metrics with measures of real-world value.

5.10 Chapter Summary

This chapter presented comprehensive experimental evaluation of our Multimodal Banglish Classroom Summarizer. Key findings include:

1. **Overall Performance:** The system achieves 73.9% Term F1, 83.6% Term Precision, and 66.5% Term Recall on the nine-video BanglaASR benchmark dataset, demonstrating practical capability for Banglish technical lecture processing.

2. **Component Contributions:** Post-processing improves F1 by 3.3% over raw Whisper output; cross-modal fusion adds an additional 2.4%. Visual information proves valuable for verification but insufficient for primary content generation.
3. **Domain Variation:** SQL/DBMS content achieves best performance (78.7% F1) due to standardized terminology and high visual availability. Python content shows more variability, with control flow topics performing well but loops proving challenging.
4. **Failure Modes:** Phonetic approximation of Bengali words (35% of errors) represents the dominant failure mode, followed by technical term confusion (25%) and repetition hallucination (18%). Visual extraction errors and temporal misalignment contribute smaller proportions.
5. **Statistical Significance:** The full pipeline significantly outperforms the baseline ($p = 0.003$) with a large effect size (Cohen’s $d = 0.96$), confirming that improvements are meaningful rather than chance variation.
6. **Baseline Establishment:** We establish reproducible baselines (68.2% baseline, 73.9% full pipeline) enabling meaningful comparison for future research on Banglish technical content.

These results demonstrate that multimodal processing of Banglish technical lectures is achievable with current pre-trained models, while highlighting substantial opportunities for improvement through domain-specific adaptation, enhanced visual processing, and larger-scale training data.

Chapter 6

Conclusion

6.1 Summary of Findings

6.2 Contributions to the Field

6.3 Recommendations for Future Work